ZisK Whitepaper[1]

ZisK Core Team

September 14, 2026

Abstract

We present ZisK, a STARK-based zero-knowledge virtual machine (zkVM) that executes programs compiled from standard instruction set architectures and produces succinct, transparent proofs of their correct execution. ZisK targets the RISC-V ISA, allowing developers to use existing Rust and C toolchains without modification. Moreover, ZisK's modular architecture enables the addition of optimized constraint systems for expensive operations such as elliptic-curve arithmetic and bulk memory access, making it practical to prove complex computations at scale.

This paper provides a precise specification of the ZisK design: its instruction set architecture, the arithmetization framework, the circuit architecture and chip design, the continuation mechanism, the recursive aggregation system, and the proving backend with its parameters.

[1]This document describes ZisK v0.16.1 [48cf7cc – 2026-20-03].
Source code available at `https://github.com/0xPolygonHermez/zisk`.

Contents

1. Introduction **2**
- 1.1. Background and Motivation . . . 2
- 1.2. ZisK . . . 2
- 1.3. Paper Overview . . . 2
- 1.4. Notation . . . 3

2. Instruction Set Architecture **3**
- 2.1. RISC-V Support and Transpilation . . . 4
- 2.2. Execution Environment . . . 5
- 2.3. State . . . 5
- 2.4. Execution Model . . . 10

3. Arithmetization Framework **12**
- 3.1. AIRs . . . 12
- 3.2. Interactions and Buses . . . 13
- 3.3. Arguments . . . 14
- 3.4. Chips . . . 15

4. ZisK Design **16**
- 4.1. Architecture Overview . . . 16
- 4.2. Chip Design . . . 18
- 4.3. Word Encoding over Goldilocks . . . 23
- 4.4. System Buses . . . 24

5. Continuations **27**
- 5.1. The Standard Approach to Continuations . . . 27
- 5.2. ZisK's Approach: Trace Splitting . . . 27
- 5.3. The Continuation Bus . . . 29
- 5.4. Parallel Proving and Recursive Aggregation . . . 30

6. Recursion **31**
- 6.1. Proof Recursion . . . 31
- 6.2. Challenge Derivation . . . 31
- 6.3. What ZisK Proves . . . 33
- 6.4. Recursive Structure . . . 33
- 6.5. Security . . . 37

7. Proving Backend **38**
- 7.1. The STARK Backend . . . 38
- 7.2. Parameters and Security Level . . . 39
- 7.3. Topology of All Proof-System Calls . . . 39

References **41**

Appendix **43**

A. Arguments **43**

B. Instruction Set **45**

1. Introduction

Recent advances in zero-knowledge (ZK) proving systems [GWC19], [Ben+18a], [DP23], [HLP24] have opened the door to a new class of cryptographic applications: systems that can verify the correct execution of an arbitrary computation without revealing its private inputs, in time and space sublinear in the original computation. These properties underpin a rapidly expanding class of applications in blockchain scalability—such as validity rollups, verifiable coprocessors, and trustless cross-chain bridges—as well as an increasingly broad range of off-chain computation protocols.

1.1. Background and Motivation.

Early ZK systems required developers to express computations directly as arithmetic circuits using frameworks such as Circom [Bel+23] or Halo2 [Ele22]. This low-level approach offers the greatest flexibility —developers can hand-tune constraint counts and choose their proof backend — but comes at a significant cost: every new computation requires custom circuit design, security review, and ongoing maintenance. The productivity model is fundamentally different from standard software development, limiting who can build ZK-enabled applications.

Zero-knowledge virtual machines (zkVMs) address this by interposing a fixed, general-purpose circuit between the developer and the proving system. The circuit verifies the correct execution of programs targeting a standard instruction set architecture (ISA), so developers can write programs in familiar languages like Rust or C and obtain proofs automatically, without writing a single constraint. The challenge for zkVM designers is then to make this general-purpose circuit as efficient as possible.

Existing zkVMs navigate a fundamental trade-off between developer experience and proving efficiency. Systems built around *ZK-native ISAs* (e.g., Cairo [GPR21] or Valida [Lit24]) define instruction sets whose semantics align naturally with finite-field arithmetic, enabling compact constraint representations but requiring developers to adopt a new programming model. Systems targeting *standard ISAs* such as RISC-V [RIS19] (e.g., RISC Zero [RIS23], SP1 [Suc24] or Nexus [Nex24]) preserve familiar toolchains and allow any RISC-V binary to run, but must constrain a general-purpose register machine whose operations —integer division, bitwise logic, memory addressing— are expensive to encode as polynomial constraints.

1.2. ZisK.

This paper introduces ZisK, a zkVM designed to make zero-knowledge proofs of general-purpose computation practical at production scale. ZisK targets the RISC-V ISA, preserving compatibility with mature toolchains such as LLVM and Rust so that programs can be proved without changes to existing development workflows. This positions ZisK in the standard-ISA camp, but with a deliberate architecture that treats proving performance as a first-class concern: rather than accepting the overhead of a general-purpose CPU circuit, ZisK uses a modular design where expensive operations are handled by dedicated, independently optimized constraint systems.

A central motivation for ZisK is *extensibility*. Application domains such as blockchain execution environments rely on a specific, known set of expensive primitives —elliptic-curve operations, hash functions, bulk memory transfers. A zkVM designed for these domains should be able to support those primitives natively, with proofs that are tight to the operation's intrinsic complexity. ZisK is structured so that such optimized components can be added as self-contained modules without modifying the rest of the system, making it practical to tailor the prover to a specific application's workload. Building on STARKs [Ben+18b] further ensures that this is achieved without a trusted setup and with post-quantum security.

1.3. Paper Overview.

Section 2 defines the ZisK ISA: the supported RISC-V subset and its transpilation, the execution environment, machine state and instruction format, and the step semantics; the full instruction set is listed in Appendix B. Section 3 presents the arithmetization framework: AIRs, bus interactions, and LogUp-based circuit satisfiability. Section 4 describes the chip architecture, the encoding of 64-bit words over Goldilocks, and the bus system. Section 5 explains trace splitting, the segments it produces, and how continuation buses chain segment boundaries. Section 6 covers the recursive aggregation tree, the recursive challenge derivation, and root-level bus balancing, and states the extractability theorem proved in the companion security analysis of the recursion [Mas26]. Section 7 fixes the proving backend and its parameters.

1.4. **Notation.**

For the remainder of the paper, we fix $\mathbb{F}$ to be a finite field of prime order p. In particular, we use the Goldilocks prime field $p = 2^{64} - 2^{32} + 1$, though this choice is not fundamental to the overall design and any appropriate prime would suffice. When needed, we identify field elements with integers in the range $[0, p)$.

We also fix a finite extension field $\mathbb{K}$ of $\mathbb{F}$, primarily for the security of the underlying proving system; concretely, ZisK targets 128-bit security. In particular, we use the third Goldilocks extension $\mathbb{F}[X]/(X^3 - X - 1)$, whose order $|\mathbb{K}| = p^3 \approx 2^{192}$ gives enough room to drive the proving system's soundness error well below 2^{-128}, while still allowing efficient arithmetic. Again, this choice is not fundamental to the design.

We denote by $\mathbb{F}[X]$ the ring of univariate polynomials over $\mathbb{F}$ and by $\mathbb{F}[X_1, ..., X_n]$ the ring of n -variate polynomials over $\mathbb{F}$. When needed, we write $\mathbb{F}^{<N}[X]$ to refer to the subset of polynomials of degree strictly less than N. We use $\mathbb{F}^{n\times m}$ to denote the set of n-by-m matrices over $\mathbb{F}$. The same notation applies to $\mathbb{K}$.

When the additive structure is the only thing that matters, we sometimes write $\mathbb{Z}_p$ for the abelian group of integers modulo p —the same underlying set as $\mathbb{F}$, viewed as a $\mathbb{Z}$-module rather than a field. Vectors over this group are denoted $\mathbb{Z}_p^n$.

We use the notation $[a, b]$ to denote the set of integers from a to b, inclusive; and $[a, b)$ to denote the set of integers from a to b, inclusive of a and exclusive of b. When $a = 1$ and $b > 1$, we write $[b]$ instead of $[1, b]$.

2. Instruction Set Architecture

The ZisK instruction set architecture (ZisK ISA) specifies the computational model that ZisK proves correct: the execution model, the memory layout, the instruction format, and the full set of operations. Because the ZisK ISA is a *virtual* ISA — not targeting any physical hardware — its design can prioritize properties that simplify zero-knowledge proof generation rather than properties that suit a particular microarchitecture.

The ZisK ISA is designed around two goals. The first is compatibility: ZisK targets the RISC-V ISA so that programs written in standard languages and compiled with existing toolchains can be run without modification. The second goal is efficiency: operations that would be expensive to constrain step-by-step in the Main chip —elliptic-curve arithmetic, hash functions, bulk memory transfers— are exposed as *precompile instructions*, each handled by a dedicated, optimized chip. This combination gives the developer accessibility of a standard ISA with the proving performance of a custom one.

The rest of this section describes how RISC-V programs are translated into the ZisK ISA (Section 2.1), the execution environment (Section 2.2), the machine state and instruction format

(Section 2.3), and the formal execution model (Section 2.4). The complete list of operations is given in Appendix B.

2.1. RISC-V Support and Transpilation.

Although programs are written and compiled as standard RISC-V binaries, ZisK does not constrain RISC-V execution directly. Before proving begins, the binary is *transpiled* off-chain into an equivalent program in the ZisK ISA, and it is the transpiled program that is committed to and executed by the VM. The transpilation is a deterministic rewriting of every RISC-V instruction into one or more ZisK instructions; it is performed by a trusted off-chain tool and is not itself part of the proof. We therefore treat the transpiler as part of the trusted computing base: every statement proved by ZisK is a statement about the committed ZisK program, and its connection to the original RISC-V binary rests on the correctness of the transpiler.

ZisK accepts 64-bit RISC-V binaries (RV64) using the following extensions:

- **RV64I**, the base integer instruction set. `fence` and `fence.i` are transpiled to no-ops, since the VM is single-threaded and has no caches; `ebreak` is also a no-op.
- **M**, all thirteen multiplication and division instructions, each mapped to a single ZisK opcode.
- **A**, the atomic instructions. Since execution is single-threaded, `lr` is a plain load, `sc` always succeeds, and every `amo` instruction is expanded into a load, the operation, and a store.
- **C**, the compressed instructions, which are transpiled exactly like their uncompressed equivalents.
- **Zicsr**, the control and status register instructions. Ordinary CSRs are plain words of the system region of memory (Table 2); a range of CSR numbers is reserved as the ports through which programs invoke precompiles and request hints (Section 2.3.10).
- **F** and **D**, the single- and double-precision floating-point instructions, which are emulated in software: floating-point loads and stores access a memory-resident floating-point register file, and every other floating-point instruction is transpiled into a call to a floating-point library that is embedded in every ZisK program.

`ecall` jumps to a trap handler that only implements the exit system call; the guest runtime uses it exclusively to terminate (Section 2.4). Any encoding outside these extensions is transpiled to the `halt` instruction, which cannot appear in a valid execution (Section 2.3.3), so a program that reaches an unsupported instruction cannot be proved.

When a RISC-V instruction at address A expands into $k > 1$ ZisK instructions, the first is placed at A and the others at intermediate addresses reserved for this purpose, and the last of them jumps to the address of the next RISC-V instruction. A RISC-V instruction therefore takes one or more ZisK steps. Table 1 shows representative translations, using the instruction fields of Section 2.3.2.

RISC-V	**ZisK instruction(s)**
`add rd, rs1, rs2`	`add` with $a =$ `REG rs1`, $b =$ `REG rs2`, store `REG rd`; `j1` $=$ `j2` $= 4$. If `rs1` or `rs2` is `x0`, a `copyb` of the other register instead.
`lw rd, imm(rs1)`	`signextend_w` with $a =$ `REG rs1`, $b =$ `IND` with displacement `imm` and width 4, store `REG rd`. `ld`, `lbu`, `lhu`, `lwu` use `copyb` instead, with the corresponding width.
`sd rs2, imm(rs1)`	`copyb` with $a =$ `REG rs1`, $b =$ `REG rs2`, store `IND` with displacement `imm` and width 8.
`beq rs1, rs2, off`	`eq` with $a =$ `REG rs1`, $b =$ `REG rs2`, no store, `j1` $=$ `off`, `j2` $= 4$. `bne` swaps the offsets; `blt`/`bge` and `bltu`/`bgeu` use `lt` and `ltu`.

RISC-V	ZisK instruction(s)
`jal rd, off`	`flag` with `store_pc`, store `REG rd`, `j1 = off`, `j2` = the size of the RISC-V instruction: jumps to `pc + off` and stores the return address `pc + j2`.
`jalr rd, imm(rs1)`	`and` with $a =$ `IMM` $2^{64}-2$, $b =$ `REG rs1`, `set_pc`, `store_pc`, store `REG rd`, `j1 = imm`: jumps to $(\text{rs1} \wedge \neg 1) + \text{imm}$. A second instruction is needed when `imm` is not a multiple of 4.
`mulh rd, rs1, rs2`	`mulh` with $a =$ `REG rs1`, $b =$ `REG rs2`, store `REG rd`.
`amoadd.w rd, rs2, (rs1)`	three instructions: `signextend_w` of the word at `rs1` into `rd`; `add_w` of the previous result and `rs2`; `copyb` of the previous result into the word at `rs1`.
`csrs 0x800, rs1`	`keccak` with $a =$ `IMM` 0, $b =$ `REG rs1` (the address of the state), no store (Section 2.3.10).

TABLE 1. Representative RISC-V to ZisK translations. Unless stated otherwise, `j2` advances to the next RISC-V instruction.

2.2. **Execution Environment.**

Programs in ZisK are executed inside a *virtual machine* (VM). The VM is a *state machine*: execution proceeds as a sequence of discrete steps, each step transforming the current *guest state* according to one instruction. The VM is hosted on a physical machine, called the *host*, that maintains an external *host state* separate from the guest. The ZisK ISA provides specific interfaces through which the guest program can read from and write to the host state.

This host/guest separation is central to ZisK's trust model. The guest program runs in a fully deterministic, sandboxed environment: every value it operates on is either derived from the program (Section 2.3.1), loaded from memory, or explicitly received from the host through a typed channel. The host can supply:

- **Inputs**. A byte string written into the input data region before execution starts and read-only thereafter.
- **Hints**. Non-deterministic values provided by the host at runtime, on the guest's request. The guest may use them, but must verify any properties it relies on.

Neither inputs nor hints are part of what the verifier sees: from the proof's point of view both are prover-chosen, and ZisK treats them as unconstrained. A guest that wants its result to be bound to its input must do so itself, for instance by exposing a hash of the input as a public output. The only data the verifier sees about an execution are the program commitment and the public outputs (Section 6.3).

The key property of this model is that *everything the guest touches is constrained by the VM's arithmetization.* Memory accesses are constrained by the memory argument; program code is validated by the ROM bus on every fetch; and public outputs become explicit proof data. The host state, by contrast, lies outside the proof.

2.3. **State.**

As defined before, the VM execution state is everything that changes from one step to the next. It is divided into the *guest state* —manipulated by the program— and the *host state* —maintained externally and supplied before or during execution. The guest state comprises the program counter `pc`, the output c of the previous instruction, the registers, and the data memory; the program itself is fixed. The host state comprises the input and the hint stream.

2.3.1. *Program ROM.*

ZisK can execute any program compiled to its ISA. A program consists of two parts:

- **The Program Code**. The program code maps 32-bit instruction addresses to ZisK instructions. It is loaded into the VM before execution begins.
- **The ROM Data**. The ROM data is an immutable data segment holding program constants (e.g., string literals, lookup tables, and static data embedded in the binary). It occupies a dedicated region of the address space and can be read but never written by the program.

Both parts are determined entirely by the program binary: they are identical across every execution of that program, regardless of the inputs provided. Once the program is fixed, neither the code nor the ROM data can change — no instruction can modify them and they play no role in the per-step state transitions of the machine. This separation of code from mutable data is the classical *Harvard architecture*: code and data live in distinct address spaces and are governed by different access rules.

From the proving system's perspective, the two parts are committed to and validated by distinct mechanisms. The program code is organised as a fixed table of instructions indexed by their addresses. Before any execution begins, this table is hashed into a *program commitment* that is part of the public statement of every proof. At every step, the Main chip performs a table lookup against this committed table —retrieving the instruction at the current program counter and verifying it belongs to the ROM. No instruction can be substituted or forged without invalidating the commitment.

The ROM data is validated through a different mechanism. Rather than a separate lookup, it is treated as a region of *immutable memory* within a dedicated address space (Section 2.3.7). Every ZisK program starts with a *boot code*, generated by the transpiler, that writes the contents of the binary's data sections —read-only and writable— into memory with ordinary store instructions, one 8-byte word at a time. Since these instructions, including their immediate values, are part of the committed program code, the initial contents of data memory are bound to the program commitment as well. The ROM data region is written once by the boot code; thereafter only reads are permitted. From the running program's point of view, reads from ROM data are indistinguishable from reads from writable RAM — the immutability is a property enforced by the arithmetization, not by the ISA. The arithmetization groups the accesses to each read-only word but does not order them in time, so the execution model treats the ROM data as present from the start: every word holds, from the first step, the value that its boot-code store writes, or zero if it has none, and that store leaves the memory unchanged (Section 2.4).

2.3.2. *Instruction Format.*

A ZisK program P is a finite partial map from 32-bit *instruction addresses* to *instructions*. Every instruction performs one operation over two input operands, producing an output and a *flag*:

$$(c, \text{flag}) = \text{op}(a, b), \quad a, b, c \in [0, 2^{64}), \text{flag} \in \{0, 1\}.$$

In essence, each instruction is a single execution step that transforms two 64-bit inputs into a 64-bit output and a boolean flag. What varies between instructions is how a and b are obtained, where c is stored, and how the program counter advances. An instruction is a record with the following fields:

- `op` $\in [0, 2^8)$ the opcode, determining the operation (Section 2.3.3).
- `a_src` $\in$ {`C`, `REG`, `MEM`, `IMM`} how a is obtained —the output c of the previous instruction, a register, the 8-byte word at a fixed address, or an immediate.
- `b_src` $\in$ {`C`, `REG`, `MEM`, `IMM`, `IND`} how b is obtained; `IND` is an *indirect* load at address a plus a displacement.

- **a_off, a_imm1** $\in [0, 2^{32})$ the register index for REG, the address for MEM, and the low and high halves of the immediate for IMM.
- **b_off, b_imm1** as for a; for IND, b_off $\in [-2^{31}, 2^{31})$ is the signed displacement.
- **ind_w** $\in \{1, 2, 4, 8\}$ the width in bytes of an IND load or store.
- **store** $\in$ {NONE, REG, MEM, IND} where the result is written —nowhere, a register, the 8-byte word at a fixed address, or ind_w bytes at address a plus a displacement.
- **store_off** the register index, the address, or the signed displacement of the store.
- **store_pc** $\in \{0, 1\}$ store the return address pc + j2 instead of c.
- **set_pc** $\in \{0, 1\}$ take the next program counter from c (indirect jump).
- **j1, j2** $\in [-2^{31}, 2^{31})$ jump offsets, j1 for a taken branch or an indirect jump and j2 otherwise. Precompile instructions reuse j1 as a static *extended argument* of the operation.
- **m32** $\in \{0, 1\}$ the operation is a 32-bit operation and only reads the low 32 bits of its operands.
- **end** $\in \{0, 1\}$ execution ends after this instruction.

Two further flags are functions of op: is_external_op, which states that the operation is proved by a chip other than Main (Section 4.2), and is_precompiled, which states that the operation reads and writes memory itself (Section 4.2.4).

Not every record of this shape is an instruction that the Main chip executes as specified, and the programs that ZisK proves are restricted accordingly.

Definition 2.1 (Valid Program). A program P is *valid* if every one of its instructions has its fields in the domains above, takes m32, is_external_op and is_precompiled from its opcode, uses only register indices in $[1, 31]$ for the sources and the store of kind REG, and sets set_pc only for an opcode whose flag is always zero.

Every program produced by the transpiler is valid. Validity is not checked by the arithmetization: the Rom chip proves whatever table is committed (Section 4.4.2), and a table that is not the ROM table of a valid program can be committed as well. A verifier therefore accepts a proof only for the program commitment of the valid program it expects.

2.3.3. *Operations.*

ZisK defines a set of opcodes, listed with their exact semantics in Appendix B, which transcribes the reference implementation[2]. They fall into the following families, each proved by one or more chips of Section 4.2:

- **Internal** (proved by Main): flag (sets the flag, used for unconditional jumps), copyb (copies b), and the halt sentinel.
- **Binary** (Binary and BinaryAdd): 64- and 32-bit comparisons, minimum and maximum, addition, subtraction and bitwise logic. Comparisons also return their result in the flag, which is how conditional branches are implemented.
- **Binary extension** (BinaryExtension): 64- and 32-bit shifts and sign extensions.
- **Arithmetic** (Arith): 64- and 32-bit multiplication (low and high halves), division and remainder.
- **Public output** (Main, bound to the public statement): pubout (Section 2.3.8).
- **Hints** (Main): fcall_param, fcall and fcall_get (Section 2.3.9).
- **Precompiles** (one chip each, see Table 4): the Keccak-f, SHA-256, Poseidon2 and BLAKE2b permutations or compression functions; 256-bit addition; 256- and 384-bit

[2]It can be found at https://github.com/0xPolygonHermez/zisk/blob/v0.16.1/core/src/zisk_ops.rs.

modular arithmetic; secp256k1, secp256r1, BN254 and BLS12-381 elliptic-curve addition and doubling; arithmetic in the quadratic extensions of the BN254 and BLS12-381 base fields; and the DMA bulk-memory operations (copy, compare, fill, and copy from the hint stream).

2.3.4. *Phantom Instructions.*

Most instructions carry a proof obligation for the operation they perform, discharged by the chip in charge of their opcode. Some instructions, however, are *phantom*: they leave the guest state unchanged apart from the trivial effect $c = b$ —acting as no-ops from the VM's perspective — but trigger side effects in the host state. These are the hint instructions `fcall_param`, `fcall` and `fcall_get` (Section 2.3.9): they appear in the main execution trace, where the Main chip proves them as plain `copyb` steps, but the host computation they trigger is not proved.

The `halt` instruction plays the opposite role: it is the sentinel into which invalid or unsupported code is transpiled (Section 2.1). The Main chip does not accept it, so it cannot appear in a valid execution trace, and a program that reaches it cannot be proved.

2.3.5. *Program Counter.*

The program counter `pc`, which is seen as a special purpose register, stores the address of the instruction currently being executed. After each instruction, `pc` advances by `j2`, unless the instruction raises the flag (a taken branch, which advances by `j1`) or sets `set_pc` (an indirect jump to c + `j1`). The exact rule is part of the step semantics of Section 2.4.

2.3.6. *Registers.*

The guest has 32 general-purpose 64-bit registers $x_0, ..., x_{31}$, following the RISC-V ABI [RIS24], accessed through the `REG` source and store kinds. Register x_0 is hardwired to zero at transpilation time: every read of x_0 is replaced by the immediate 0 and every write to it is dropped, so the VM's register state is $x_1, ..., x_{31}$. The system region of memory (Table 2) reserves 8-byte slots for the registers at its start, but these slots are not aliased with the registers: registers are only accessed through the `REG` source and store kinds, and ZisK handles them separately from the rest of memory for performance reasons (Section 4.2.1.1).

2.3.7. *Data Memory.*

Data memory is a *random access memory (RAM)* which supports read and write operations. It is separate from the program ROM and is used for mutable state during execution. The ZisK ISA defines a 32-bit byte address space for data memory, but the native unit of access is a 64-bit word (8 bytes). Memory cells are aligned to 8-byte boundaries, so valid addresses are multiples of 8. ZisK supports sub-word and unaligned accesses (1, 2, 4 or 8 bytes at any address) by decomposing them into aligned 8-byte operations (Section 4.2.2).

To support execution of programs with different data assumptions and properties within the same architecture, we separate data memory into distinct regions, each with its own access semantics. Table 2 summarises the address space layout and the semantics of each region; addresses outside these regions are invalid.

Region	**Address Range**	**Semantics**
Input Data	`[0x4000_0000, 0x8000_0000)`	[1GB] Read-only. The first word, `0x4000_0000`, is the *free-input port* through which hints are delivered (Section 2.3.9); the input starts at `0x4000_0008`.

Region	Address Range	Semantics
ROM Data	`[0x8000_0000, 0x8800_0000)`	[128MB] Program constants, read-only
Writable RAM	`[0xA000_0000, 0xA001_0000)`	[64KB] System region: register slots, UART, control and status registers, floating-point registers
	`[0xA001_0000, 0xA002_0000)`	[64KB] Output data: public outputs written during execution
	`[0xA003_0000, 0xC000_0000)`	[≈512MB] Program memory: general-purpose read/write RAM, including the stack and heap

TABLE 2. Memory layout of the ZisK ISA. The address space is divided into three main regions: input data, ROM data, and writable RAM. The 64KB between the output data and the program memory are unused.

2.3.8. *Public Outputs.*

The guest program can expose up to 32 values of 64 bits (2048 bits in total) as *public outputs*. During execution, the guest writes them to the output data region (Table 2) with ordinary stores. When the guest finishes, an *epilogue* generated by the transpiler reads the 32 words of that region back and issues one `pubout` instruction per word, with $a = i \in [0, 32)$ the index of the output and b its value. The `pubout` instructions are what binds the proof's public statement to the execution: the public outputs of the statement are exactly these 32 words (Section 6.3), and nothing else about the input or the memory.

2.3.9. *Inputs and Hints.*

The host state maintains two streams through which the outside world communicates with the guest. The *input* is a byte string, of length a multiple of 8, that is loaded into the input data region at address `0x4000_0008` before execution begins; the guest reads it with ordinary loads. The *hint stream* provides non-deterministic values at runtime. To obtain hints, the guest passes parameters to the host with `fcall_param`, requests a host computation with `fcall` —whose operand a identifies the function— and then reads the result one word at a time with `fcall_get`, each of which reads the free-input port at `0x4000_0000`. Every read of the free-input port may return a different value: it is the only source of non-determinism in a step. Any value received from the host, input or hint, is untrusted until the guest has verified it.

2.3.10. *Precompile Calls.*

Precompiles are invoked through a calling convention based on CSR instructions, which are free for this purpose since ZisK has no privileged architecture. The caller writes the operands of the operation to memory with ordinary instructions —typically a *parameter block* of 8-byte-aligned words holding the operands or pointers to them— and then executes `csrrs x0, csr, rs1` with `rs1` holding the address of the block. The transpiler maps CSR numbers `0x800` to `0x819` to the precompile opcodes, and turns the CSR instruction into a single ZisK instruction with that opcode, $a = 0$ and $b =$ `REG rs1`. The precompile reads its operands from memory and writes its results back to memory, as described for each opcode in Appendix B; only `add256` additionally returns a value in c, its carry out, which is stored in the destination register of the CSR instruction.

The hint instructions use the same mechanism: CSR numbers `0x8C0` to `0x8DF` encode `fcall` with the function identifier in the CSR number and immediate, `0x8F0` to `0x8FF` encode `fcall_param`, and `0xFFE` encodes `fcall_get`.

2.4. Execution Model.

The execution model defines how the VM transitions between states, what constitutes a valid execution, and which parts of an execution are exposed to the verifier.

Memory as a function. We model data memory as a function $\mathrm{mem} : [0, 2^{32}) \to [0, 2^8)$ from byte addresses to bytes. For a width $w \in \{1, 2, 4, 8\}$ and an address x we write $\mathrm{mem}[x]_w := \sum_{i=0}^{w-1} 2^{8i} \cdot \mathrm{mem}(x + i)$ for the little-endian w-byte read, and $\mathrm{mem}[x]_w \leftarrow v$ for the memory that stores the w low-order bytes of v at $x, ..., x + w - 1$ and agrees with mem elsewhere.

Guest state. Since the program is fixed before execution begins, the *guest state* at step t is

$$S_t = (\mathrm{pc}_t, c_t, \boldsymbol{x}_t, \mathrm{mem}_t),$$

comprising the program counter, the output of the previous instruction, the registers $\boldsymbol{x}_t = (x_1, ..., x_{31})$, and the data memory. The host state $H = (\mathrm{in}, \mathrm{hint})$ is supplied externally.

Initial state. Execution begins at a designated boot address boot address $\mathrm{pc}_{\mathrm{BOOT}}$:

$$S_0 = (\mathrm{pc}_{\mathrm{BOOT}}, 0, \mathbf{0}, \mathrm{mem}_0),$$

where mem_0 holds the input in the input data region, the values that the ROM data policy below prescribes in the ROM data region, and zero everywhere else. The boot code then initialises the data sections of the program (Section 2.3.1), sets up the arguments of the guest's entry point and calls it.

Step. Let $\iota = P[\mathrm{pc}]$ be the instruction at the current program counter; execution fails if `pc` is not an address of P. Given the state $S = (\mathrm{pc}, c, \boldsymbol{x}, \mathrm{mem})$, one step proceeds as follows (we write $x_0 = 0$).

1. *Operands.* The operands are

$$a := \begin{cases} c & \text{if } \iota.\mathrm{a_src} = \texttt{C} \\ x_{\iota.\mathrm{a_off}} & \text{if } \iota.\mathrm{a_src} = \texttt{REG} \\ \mathrm{mem}[\iota.\mathrm{a_off}]_8 & \text{if } \iota.\mathrm{a_src} = \texttt{MEM} \\ \iota.\mathrm{a_off} + 2^{32} \cdot \iota.\mathrm{a_imm1} & \text{if } \iota.\mathrm{a_src} = \texttt{IMM} \end{cases} \qquad b := \begin{cases} c & \text{if } \iota.\mathrm{b_src} = \texttt{C} \\ x_{\iota.\mathrm{b_off}} & \text{if } \iota.\mathrm{b_src} = \texttt{REG} \\ \mathrm{mem}[\iota.\mathrm{b_off}]_8 & \text{if } \iota.\mathrm{b_src} = \texttt{MEM} \\ \iota.\mathrm{b_off} + 2^{32} \cdot \iota.\mathrm{b_imm1} & \text{if } \iota.\mathrm{b_src} = \texttt{IMM} \\ \mathrm{mem}[a + \iota.\mathrm{b_off}]_{\iota.\mathrm{ind_w}} & \text{if } \iota.\mathrm{b_src} = \texttt{IND} \end{cases}$$

   where an indirect access requires $a < 2^{32}$. The k-th read of the free-input port in the step returns the k-th hint word of the step instead of the memory contents.
2. *Operation.* $(c', \mathrm{flag}) := \iota.\mathrm{op}(a, b)$, as specified in Appendix B. Precompile opcodes additionally read and write memory, as specified there; we write $\mathrm{mem}^{\mathrm{op}}$ for the memory after this effect ($\mathrm{mem}^{\mathrm{op}} = \mathrm{mem}$ for all other opcodes).
3. *Store.* The stored value is $v := \mathrm{pc} + \iota.\mathrm{j2}$ if $\iota.\mathrm{store_pc} = 1$ and $v := c'$ otherwise. The new registers and memory are those of $\mathrm{mem}^{\mathrm{op}}$ and $\boldsymbol{x}$, updated by $x_{\iota.\mathrm{store_off}} \leftarrow v$ if $\iota.\mathrm{store} = \texttt{REG}$ (and $\iota.\mathrm{store_off} \neq 0$), by $\mathrm{mem}^{\mathrm{op}}[\iota.\mathrm{store_off}]_8 \leftarrow v$ if $\iota.\mathrm{store} = \texttt{MEM}$, and by $\mathrm{mem}^{\mathrm{op}}[a + \iota.\mathrm{store_off}]_{\iota.\mathrm{ind_w}} \leftarrow v$ if $\iota.\mathrm{store} = \texttt{IND}$.
4. *Control flow.* The next program counter is

$$\mathrm{pc}' := \begin{cases} (c' \bmod 2^{32}) + \iota.\mathrm{j1} & \text{if } \iota.\mathrm{set_pc} = 1 \\ \mathrm{pc} + \iota.\mathrm{j1} & \text{if } \iota.\mathrm{set_pc} = 0 \wedge \mathrm{flag} = 1 \\ \mathrm{pc} + \iota.\mathrm{j2} & \text{if } \iota.\mathrm{set_pc} = 0 \wedge \mathrm{flag} = 0 \end{cases}$$

An instruction never has both ι.set_pc $= 1$ and flag $= 1$; in particular, precompile instructions have flag $= 0$ and set_pc $= 0$, so their extended argument in `j1` never reaches the program counter.

The new state is $S' = (\mathrm{pc}', c', \boldsymbol{x}', \mathrm{mem}')$. We write $S' = \mathrm{Step}_P(S, h)$, where h is the sequence of hint words that the step reads, in order. Given the hints, the step is deterministic, and the whole execution is a function of the program, the input and the hint stream. We write a_t and b_t for the operands of the step from S_t.

Valid steps. The step from $S = (\mathrm{pc}, c, \boldsymbol{x}, \mathrm{mem})$ with hints h is *valid* if:

- pc is an address of P, and $P[\mathrm{pc}]$ is not `halt`.
- Every memory access it makes covers w bytes $x, ..., x + w - 1$ that lie in one region of Table 2.
- It writes nothing to the input data region.
- It reads exactly $|h|$ words from the free-input port.

An execution moreover respects the *ROM data policy* if every write to the ROM data region leaves the memory unchanged, every 8-byte word of the region is written at most once, and every word of the region that is never written is zero in mem_0. The contents of the region are thus fixed from the start by the stores of the boot code, which leave them unchanged, and a read of the region returns the same value whenever it happens.

Terminal condition. Every ZisK program contains, at the exit address $\mathrm{pc}_{\mathrm{END}}$, an *exit instruction* with the `end` field set; the only other instructions with `end` set are `halt` sentinels. The guest terminates by returning from its entry point or by the exit system call; in both cases control passes to the epilogue, which emits the public outputs (Section 2.3.8) and jumps to $\mathrm{pc}_{\mathrm{END}}$. A T-step execution is *successful* if $\mathrm{pc}_T = \mathrm{pc}_{\mathrm{END}}$ and $c_T = 0$, every step before T is valid, and the execution respects the ROM data policy. Execution *fails* otherwise: on an undefined program counter, an access outside the regions of Table 2, a write to the input data region, a write that changes the ROM data region or a second write to one of its words, the `halt` sentinel, or if the program counter never reaches $\mathrm{pc}_{\mathrm{END}}$. Only successful executions are provable, and ZisK bounds their length by $T \leq 2^{36}$.

Public outputs. The public outputs are not a separate component of the state. For $i \in [0, 32)$, the i-th public output of the execution is the operand b of the unique step t that executes `pubout` with $a = i$; since the epilogue reads them from the output data region, this is the word $\mathrm{mem}_t[$ `0xA001_0000`$+8i]_8$ at the end of the guest's execution.

The execution relation. What a ZisK proof claims is a relation between the external statement —the program commitment and the public outputs— and a whole execution. Its witness contains the program itself, which the statement binds only through its commitment, the input, the hints and the sequence of states. We write $\mathrm{RomRoot}(P)$ for the Merkle root of the Rom chip's cached columns for the program P (Section 4.4.2), which the setup computes from P.

Definition 2.2 (Execution Relation). The statement is $\boldsymbol{p}_{\mathrm{ext}} = (\mathrm{root}_{\mathrm{ROM}}, \mathrm{out}_0, ..., \mathrm{out}_{31})$, with $\mathrm{root}_{\mathrm{ROM}} \in \mathbb{F}^4$ and $\mathrm{out}_i \in [0, 2^{64})$, and the witness is $w_{\mathrm{exec}} = \Big(P, \mathrm{in}, (h_t)_{t<T}, (S_t)_{t\leq T}\Big)$. Then $(\boldsymbol{p}_{\mathrm{ext}}; w_{\mathrm{exec}}) \in \mathcal{R}_{\mathrm{exec}}$ if and only if:

1. *Program.* P is valid (Definition 2.1) and $\mathrm{RomRoot}(P) = \mathrm{root}_{\mathrm{ROM}}$.
2. *Initial state.* $S_0 = (\mathrm{pc}_{\mathrm{BOOT}}, 0, \mathbf{0}, \mathrm{mem}_0)$, where mem_0 holds in from address `0x4000_0008` and is zero outside the input data and ROM data regions.
3. *Steps.* For every $t < T$, the instruction $P[\mathrm{pc}_t]$ is defined, the step from S_t with hints h_t is valid, and $S_{t+1} = \mathrm{Step}_P(S_t, h_t)$.
4. *ROM data.* The execution respects the ROM data policy.

5. *Termination.* $\mathrm{pc}_T = \mathrm{pc}_{\mathrm{END}}$, $c_T = 0$ and $T \leq 2^{36}$.
6. *Public outputs.* Every step $t < T$ that executes `pubout` has $a_t < 32$ and $b_t = \mathrm{out}_{a_t}$, and for every $i < 32$ exactly one step executes `pubout` with $a_t = i$.

The statement contains neither the input nor the hints, and the relation constrains them only through the execution they produce: a guest that wants its outputs bound to its input must expose, for instance, a hash of the input among them. The relation concerns the committed ZisK program; its connection to the original RISC-V binary is the transpiler correctness of Section 2.1, which is assumed. The companion security analysis [Mas26] proves that an accepting final proof for a statement $\boldsymbol{p}_{\mathrm{ext}}$ whose program commitment is that of a valid program yields, except with a stated probability, a witness of $\mathcal{R}_{\mathrm{exec}}$.

3. Arithmetization Framework

A proving system is naturally divided into two layers. The *frontend* takes a program and produces a structured mathematical representation — typically a system of polynomial constraints — that captures exactly what it means for the computation to be correct. The *backend* then takes that representation and produces a cryptographic proof of its satisfiability. Concretely, the backend performs a one-time *setup* that yields a pair $(\mathsf{pk}, \mathsf{vk})$: a *proving key* used to generate proofs of the frontend's constraint system, and a *verification key* that summarises the constraint system in a form the verifier can check against. Verification of a proof π against a public statement x is then the deterministic procedure $\mathbf{V}(\mathsf{vk}, x, \pi) \in \{\mathrm{accept}, \mathrm{reject}\}$. The two layers are decoupled by design: a single frontend can target different backends, and improvements to either do not require redesigning the other.

ZisK's *arithmetization framework* is its frontend. The choice of arithmetization is consequential: it determines how computations are structured, what operations are native, and how efficiently the backend can prove them. ZisK's arithmetization is designed to be modular and flexible, capturing a wide range of computations while optimizing for the patterns that arise most frequently in zkVM execution.

The frontend is based on a generalization of the *algebraic intermediate representation (AIR)* framework [Ben+18b], which specifies polynomial constraints over a witness matrix; coupled with arguments such as LogUp [Hab22], which extends AIR with logarithmic derivative-based lookups. A single AIR captures the behavior of one logical unit (e.g., the CPU, the ALU, or the memory). Since realistic computations involve multiple interacting units, we group AIRs together and define *interactions* between them —messages exchanged through shared communication channels called *buses*. This modular design has several advantages:

- **Separation of concerns**: Each AIR can be designed, implemented, and tested independently.
- **Composability**: AIRs can be reused across different computations or combined in different configurations.
- **Scalability**: Different AIRs can have different sizes, allowing resources to be allocated based on actual usage.
- **Backend independence**: The arithmetization can be paired with any compatible proving system.

The remainder of this section describes these concepts. Their formal definitions are those of the companion security analysis [Mas26], which this section follows.

3.1. AIRs.

An *algebraic intermediate representation* (AIR) [Ben+18b] is a finite set of polynomial *constraints* over the columns of a trace, each read at a fixed set of row offsets, and over a vector of *AIR values*: field elements attached to the instance rather than to a row. A *witness* is a trace $\boldsymbol{T} \in \mathbb{F}^{N \times W}$, of a height N that is a power of two, together with its AIR values; it *satisfies* the AIR if every constraint vanishes at every row, row indices being taken modulo N. AIR values are a non-standard but necessary extension: they express instance-wide constants, such as the publics or the index of a segment, as first-class citizens of the constraint system rather than as workarounds encoded in the trace. The constraints of a ZisK AIR have coefficients in the Goldilocks field $\mathbb{F}$, and its trace and values lie in $\mathbb{F}$; the proving system extends them over the cubic extension $\mathbb{K}$ to realise the interactions (Section 3.3.1).

Not all columns in a trace have the same lifecycle. Some are fully determined by the program before any execution begins; others depend on the specific inputs provided; and others are intermediate values the prover computes but need not reveal. Distinguishing these cases allows the proving system to avoid redundant work —a column that is the same across all executions of a program need only be committed once— and to be precise about what the verifier is entitled to see. Trace columns are therefore partitioned into three categories according to when they are determined and who can access them:

- **Fixed**: Determined at setup time, independent of any particular execution. Fixed columns are shared with the verifier as part of the AIR's description; when an AIR has fixed columns, the trace height N is also fixed at setup. Typical uses include lookup tables and instruction encodings.
- **Cached**: Computed by the prover and committed once, then reused across multiple executions of the same program. Cached columns capture data that depends on the program but not on its inputs —such as the ROM instruction trace— avoiding redundant recomputation.
- **Witness**: Computed fresh for each execution and not revealed to the verifier. These columns carry the execution-specific state: operation values, memory accesses, and all intermediate values produced during a run.

3.2. Interactions and Buses.

A single AIR is self-contained: its constraint polynomials relate columns within the same trace and cannot reference columns of another AIR. Yet realistic computations are inherently cross-cutting —the CPU must read values that the memory has written, and a hash precompile must receive inputs from and return outputs to the main execution trace. To express these relationships without merging all AIRs into one monolithic constraint system, we introduce *interactions*: row-level messages that one AIR sends and another receives, mediated by a shared channel called a *bus*. An interaction specifies which values are communicated at each row and with what multiplicity; consistency across AIRs is then enforced by requiring that every bus is *balanced* —the total signed flow of messages sums to zero.

An *interaction* of an AIR is a tuple $(b, m, \boldsymbol{e})$ of a *bus identifier* $b \in \mathbb{F}$, a *multiplicity* m and a *message* $\boldsymbol{e} = (e_1, ..., e_\ell)$, the last two polynomials in the variables of the AIR. Evaluated on a witness at row r, they give $m(r) \in \mathbb{F}$ and $\boldsymbol{e}(r) \in \mathbb{F}^\ell$, and the interaction puts the message $\boldsymbol{e}(r)$ on the bus b with multiplicity $m(r)$: positive when the AIR sends it, negative when it receives it. A *bus* is the set of interactions that share a bus identifier, and all the interactions on a bus have the same message length.

The fundamental consistency condition is that every bus is *balanced*: over all the AIR instances of a proof, together with the *global messages* that the system adds as a function of the public

statement (Section 6.3), the signed multiplicities of every message on every bus add up to zero. Nothing is sent that is not received, and vice versa. As stated, the condition ranges over an exponentially large set of candidate messages and is impractical to check directly; arguments and LogUp turn it into a small set of polynomial constraints. Interactions and bus balancing define the communication structure, but not what semantic relationship is enforced: that is the role of arguments, and the meaning of each bus of ZisK is given in Section 4.4.5.

3.3. **Arguments.**

Arguments are the semantic tools used to specify what consistency must hold across bus interactions: a permutation argument asserts that two multisets are equal or a lookup argument asserts containment in a table. Each argument type is realized as a pair of interactions on a shared bus —one AIR sending with positive multiplicity, another receiving with negative multiplicity— so that bus balancing becomes equivalent to the desired relationship holding. A detailed description of the specific arguments used in ZisK is provided in Appendix A.

Bus balancing is therefore the unified consistency condition underlying all arguments. However, checking it naively —summing over every row of every AIR— requires the verifier to process the full witness, defeating succinctness. We use *LogUp* [Hab22] to reduce bus balancing to a small set of polynomial constraints that the backend can verify efficiently. The reduction proceeds in two steps: first, compress each multi-element message into a single field element; then, express the balancing sum as a polynomial identity checkable at a random location.

3.3.1. *LogUp Reduction.*

The key insight of LogUp is that multiset equality can be recast as a rational identity over a random challenge. Two multisets S and R (with multiplicities) are equal if and only if, for a *challenge* α:

$$\sum_{s \in S} \frac{1}{\alpha + s} = \sum_{r \in R} \frac{1}{\alpha + r}$$

Here, by *challenge* we refer to a random field element sent by the verifier after the prover has committed to its witness; because the prover cannot predict it before committing, the challenge breaks any attempt to craft a witness that satisfies the identity by accident. In the non-interactive setting, challenges are derived deterministically from the prover's commitments via a collision-resistant hash (Fiat-Shamir transform [FS87]), preserving the same guarantee without interaction.

To apply this idea to bus balancing, we first need to compress each multi-element message $\boldsymbol{e}(r)$ into a single field element. This compression itself uses a second challenge $\gamma \in \mathbb{K}$. The two challenges are derived together: the pair $\chi = (\alpha, \gamma) \in \mathbb{K}^2$, which we call the *key* of the bus argument, is a single Fiat-Shamir output that becomes available only after the witness has been committed (Section 6.2).

Definition 3.1 (Compressed Interaction Value). Given an interaction $(b, m, \boldsymbol{e})$ of length ℓ and a challenge $\gamma \in \mathbb{K}$, the *compressed value* at row r is:

$$\hat{e}(r) = b + \sum_{j=1}^{\ell} \gamma^j \cdot e_j(r)$$

The bus identifier b is included to prevent cancellation across different buses.

After compression, LogUp defines an auxiliary column to account for the sum. For each AIR $\mathcal{C}_i$ with trace of length N_i and set of interactions $\mathcal{I}_i$, the contributions of all its interactions to the bus are accumulated row by row into a *partial sum* S_i, defined by the recurrence:

$$S_i(r) = \begin{cases} \displaystyle\sum_{(b,m,\boldsymbol{e})\in\mathcal{I}_i} \frac{m(1)}{\alpha+\hat{e}(1)}, & \text{if } r=1 \\ \displaystyle S_i(r-1) + \sum_{(b,m,\boldsymbol{e})\in\mathcal{I}_i} \frac{m(r)}{\alpha+\hat{e}(r)}, & \text{if } r>1 \end{cases}$$

The final value $s_i = S_i(N_i)$ is the AIR's total signed contribution to the bus. We call it the *bus contribution* of the witness under the key χ and write

$$s_i = \mathrm{BusCont}_{\chi}(\mathcal{I}_i, (\boldsymbol{T}_i, \boldsymbol{a}_i)).$$

Global bus balancing then reduces to a single scalar constraint:

$$\sum_{i=1}^{M} s_i + s_{\mathrm{glob}} = 0,$$

where s_{glob} is the same LogUp sum taken over the *global messages*: messages that no AIR emits but that the system as a whole adds to some buses, such as the anchors that open and close the continuation chains (Section 5.3). They are determined by the public statement, and Section 6.3 lists them. The bus contribution is computed in the extension field $\mathbb{K}$ of the challenges; the exact columns and constraints that compute it are those of [Mas26], which also accounts for the keys at which a denominator vanishes. This constraint is enforced at the aggregation level: individual AIR proofs expose s_i as a public output, and the recursion verifies that they sum, together with s_{glob}, to zero (see Section 6).

3.4. **Chips.**

A *chip* is an AIR together with its set of interactions. ZisK is a fixed family of chips

$$\mathfrak{A} = \{(\mathcal{C}_\tau, \mathcal{I}_\tau)\}_{\tau\in[M]},$$

listed in Table 4, whose index τ is the *type* of the chip. When the design of Section 4 groups several AIRs under one name, such as the three AIRs of the Memory chip, the formal objects remain the AIRs, each with its interactions.

Each chip carries its *own* trace height N_τ. This is a deliberate design choice and one of the main reasons for organising a computation as a family of chips rather than as a single monolithic AIR. Different AIRs typically encode different units of work with different natural sizes —one AIR may scale with the length of the program being proved, another with the number of times a particular operation is invoked, and another with the size of a fixed lookup table— and there is no reason for these sizes to coincide. Forcing all AIRs to share a common height would either pad the cheap ones with empty rows or truncate the expensive ones below their needs, both of which translate directly into wasted prover work. Allowing per-AIR heights instead lets each AIR be sized to its true workload, so the prover only pays for what each unit actually does. This independence is also what makes per-AIR mechanisms such as continuations (Section 5) possible without affecting the rest of the system.

A collection of witnesses, with any number of witnesses of each type, *satisfies* the family if every witness satisfies the AIR of its type and every bus is balanced, counting the global messages of the public statement. This is what the recursion of Section 6 establishes about the AIR instances of a proof, and the security analysis [Mas26] states it as a relation.

4. ZisK Design

We describe the architecture of chips that constitutes the ZisK zkVM. The previous sections established the building blocks: the ISA defines what programs can run (Section 2), and the arithmetization framework defines how computations are represented as polynomial constraints (Section 3). Here we describe how ZisK instantiates that framework into a concrete system.

Definition 4.1 (ZisK). Given the ZisK ISA, the *ZisK zkVM* is the collection of:

1. A host execution environment capable of executing every ZisK instruction.
2. A family of chips $\mathfrak{A}$ (Section 3.4) that constrains a finite sequence of guest state transitions to be valid with respect to a public commitment to a program ROM and an initial guest state, and exposes the final guest public outputs as public values.
3. Methods to generate the trace matrices from the guest state transitions necessary to generate a proof for $\mathcal{C}$.

An AIR instance covers only a finite sequence of state transitions, whereas program execution is a priori unbounded. Fixing a single large trace height upfront is impractical: the height would have to be chosen conservatively enough to accommodate every possible program, inflating proof costs even for short executions, while still failing for programs that exceed it. *Continuations* resolve this tension by organizing and partitioning the execution trace into fixed-height segments, each proved independently. We describe how ZisK implements this mechanism in Section 5.

4.1. Architecture Overview.

ZisK is organized into *chips*, where each chip encapsulates an AIR, with its interactions, together with the methods to generate its trace matrix. Chips fall into two categories:

- **Instruction executor**: Owns the execution logic for a subset of the ZisK ISA. Each instruction is handled by exactly one chip, though a chip may handle multiple related instructions.
- **Table**: Provides auxiliary functionality —lookup tables, range checks, or cached computation results— shared by one or more instruction executors.

Chips communicate exclusively through *system buses*: shared channels through which chips exchange messages. The set of system buses used in ZisK is summarized in Table 3. While there are globally shared buses, each chip may expose interfaces for communicating with other particular chips. This modular design means that new instructions can be supported by introducing a new chip and connecting it to the appropriate buses, without modifying any existing chip.

Bus	Purpose
Operation Bus	Routes operations requiring dedicated constraints from some chip to other specialized chips
ROM Bus	Verifies each instruction fetch against the committed program ROM
Memory Bus	Enforces read/write consistency across all memory accesses
Continuation Buses	Carries chip state across segment boundaries
Table Buses	Connects instruction executor chips to shared lookup tables

Table 3. Bus system in ZisK. Each bus serves a specific purpose in the architecture, enabling modular communication between chips and supporting the overall proof structure.

An instruction executor may delegate part of its computation to other chips via a bus, which may in turn query a table chip, and so on. This creates a layered dependency structure, illustrated in Figure 1: each chip proves only its own local constraints, and bus balancing composes the correctness arguments upward into a global proof.

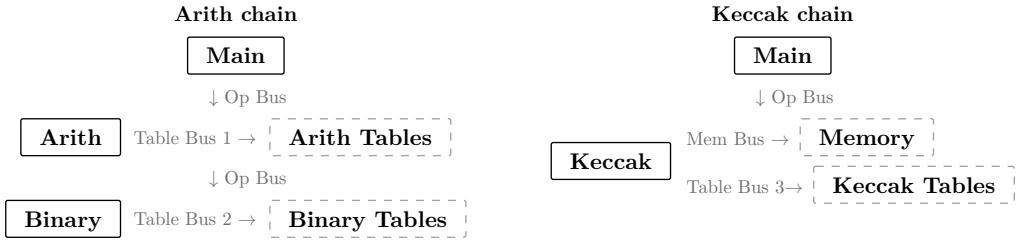


FIGURE 1. Two example chip dependency chains in ZisK. Solid borders denote instruction executor chips; dashed borders denote table chips. **Left**: the Main chip dispatches an arithmetic operation to Arith via the Operation Bus; Arith queries its own tables and further delegates to Binary, which queries its own tables. **Right**: Main dispatches a cryptographic operation to Keccak; the precompile simultaneously reads its inputs and writes its outputs via the Memory Bus and queries its own tables. In both cases each chip proves only its own local constraints, and bus balancing composes the arguments into a global proof.

4.1.1. *Chip Inventory.*

Table 4 lists the family of chips $\mathfrak{A} = \{(\mathcal{C}_\tau, \mathcal{I}_\tau)\}_{\tau \in [M]}$ of Section 3.4: every AIR of ZisK together with its trace height N_i, its category, and what it proves. There are $M = 35$ AIRs. Several chips are realised by more than one AIR —the Memory chip by one AIR per address region, the DMA precompile by twelve AIRs specialised by operation— so the formal objects are the AIRs, and "chip" is a grouping used for exposition. Recall that the opcode handled by each instruction executor is listed in Appendix B.

The family is fixed: every AIR is part of every ZisK proof's setup. What varies between executions is the number of *instances* of each AIR, i.e. the number of segments into which its trace is split (Section 5). An execution uses $\lceil n_i/N_i \rceil$ instances of AIR i, where n_i is the number of rows its workload requires, and none at all of an AIR whose operations never occur. Main and Mem are the exception: their continuation chains are anchored unconditionally (Section 5.3), so every execution instantiates each of them at least once.

AIR	$\log_2 N_i$	**Category**	**Proves**
Main	22	executor	one instruction step per row: fetch, operand resolution, dispatch, store, control flow, registers
Rom	22	table	the program code, as a lookup table on the ROM Bus (cached columns, committed per program)
Mem	22	memory	sorted access trace of the writable RAM region
RomData	21	memory	sorted access trace of the read-only ROM data region
InputData	21	memory	sorted access trace of the input data region
MemAlign	21	memory	sub-word and unaligned accesses, decomposed into aligned ones
MemAlignByte, MemAlignReadByte, MemAlignWriteByte	22	memory	single-byte accesses (read and write, read only, write only)

AIR	$\log_2 N_i$	Category	Proves
Arith	21	executor	64- and 32-bit multiplication, division and remainder
Binary	22	executor	comparisons, minimum and maximum, addition and subtraction, bitwise logic
BinaryAdd	22	executor	64-bit addition (a cheaper alternative to Binary for the most frequent operation)
BinaryExtension	22	executor	shifts and sign extensions
Add256	20	precompile	256-bit addition with carry
ArithEq	20	precompile	256-bit modular arithmetic; secp256k1, secp256r1 and BN254 curve operations; $\mathbb{F}_{p^2}$ arithmetic over the BN254 base field
ArithEq384	20	precompile	384-bit modular arithmetic; BLS12-381 curve operations; $\mathbb{F}_{p^2}$ arithmetic over the BLS12-381 base field
Keccakf	17	precompile	Keccak-f[1600] permutation
Sha256f	18	precompile	SHA-256 compression function
Poseidon2	17	precompile	Poseidon2 permutation of width 16
Blake2br	18	precompile	BLAKE2b round function
Dma, DmaMemCpy, DmaInputCpy	21	precompile	DMA front-ends: decode a bulk memory copy, compare or fill into its phases
Dma64Aligned, Dma64AlignedInputCpy, Dma64AlignedMemSet, Dma64AlignedMem, Dma64AlignedMemCpy	21	precompile	the 8-byte-aligned body of a bulk operation
DmaUnaligned	21	precompile	the body of a bulk operation whose source and destination are not equally aligned
DmaPrePost, DmaPrePostMemCpy, DmaPrePostInputCpy	21	precompile	the partial head and tail words of a bulk operation
SpecifiedRanges	20	table	every range-check table of ZisK
VirtualTable0, VirtualTable1	21	table	the fixed lookup tables of the executors and precompiles (binary, binary extension, arithmetic, Keccak, MemAlign and DMA tables, dual-byte ranges) and the frequent-operation caches of Section 4.2.5, packed into two AIRs

TABLE 4. The $M = 35$ AIRs of ZisK. Rows that list several AIRs share the trace height shown.

Now we delve into the details of the design of the ZisK chips (Section 4.2) and bus system (Section 4.4).

4.2. **Chip Design.**

We now turn to the chips themselves: the AIRs that, together with the bus system, implement the ZisK zkVM. The presentation roughly follows the dependency structure of Figure 1, starting from the central spine and moving outwards. We begin with the *Main* chip (Section 4.2.1), which materializes one row per instruction step and orchestrates the rest of the system: it fetches instructions, resolves operands, dispatches operations to specialized sub-chips, and writes back results. We then describe the *Memory* chip (Section 4.2.2), which centralizes the memory consistency argument that almost every other chip ultimately relies on. The remaining categories cover what the

Main delegates: *base-operation* chips (Section 4.2.3) handle classes of simple operations —binary, arithmetic, comparisons— through dedicated AIRs and lookup tables; *precompiles* (Section 4.2.4) handle expensive or memory-intensive primitives such as elliptic-curve arithmetic, hash functions, and bulk transfers, each as an independent instruction executor with its own internal layout; and *frequent-operation caches* (Section 4.2.5) commit pre-computed results for the most commonly queried inputs of base operations, replacing constraint checking with a direct lookup wherever possible.

4.2.1. *The Main.*

ZisK uses a *Main* chip as the central execution spine. Each row of the Main AIR corresponds to one instruction step and its trace materializes the full execution transcript. Recalling the notation from Section 2.3.2, the Main chip is responsible for:

1. **Instruction fetch**: At each step, the Main chip reads the instruction at the current program counter from the program ROM via the ROM Bus (Section 4.4.2).
2. **Operand resolution**: The input operands a and b are resolved according to their source modes (immediate, memory, or indirect), with memory reads constrained through the Memory Bus.
3. **Operation dispatch**: Operations that are simple enough are computed directly in the Main chip (e.g., no-ops, jumps, and control flow). Operations requiring dedicated constraint systems —binary operations, multiplications, cryptographic primitives— are *delegated*: the Main chip sends the inputs and receives back the result (c, flag) via the Operation Bus, and a specialized sub-chip proves correctness.
4. **State update**: The output c is written to its destination via the Memory Bus or directly in the Main trace.

4.2.1.1. *Registers.*

Although the ZisK ISA addresses the 32 general-purpose registers within the same numerical address space as data memory (Section 2.3.7), ZisK does *not* route register accesses through the Memory chip. Forcing every register read and write into the sorted memory trace would inflate it —registers are by far the most frequently accessed locations in any execution— and would couple the cost of register access to the cost of generic RAM. Instead, the register region is carved out of the Memory chip's domain and handled by the Main chip itself, with a *linked-step argument* on the Memory Bus whose messages are both sent and received by Main.

Register accesses use a dedicated operation tag `REG` on the Memory Bus and the bare register index $i \in [1, 31]$ as address, so they can never be confused with a RAM access. For every access to register i at memory step t_{mem} (Section 4.2.1.2) with value v, the Main row also carries the memory step t_{prev} and value v_{prev} of the previous access to the same register, and it

- consumes the message $\big(\texttt{REG}, i, t_{\text{prev}}, 8, v_{\text{prev}}\big)$, which certifies that the previous access left v_{prev} in the register;
- emits the message $(\texttt{REG}, i, t_{\text{mem}}, 8, v)$, which the next access to the register will consume;
- range-checks $t_{\text{mem}} - t_{\text{prev}} - 1 \in [0, 2^{24})$, so the accesses to each register form a chain strictly increasing in time.

For a read $v = v_{\text{prev}}$; for a write v_{prev} is a free column. The chain of every register is opened once per execution by a global term that emits $(\texttt{REG}, i, 0, 8, 0)$ for $i \in [1, 31]$ —registers start at zero— and closed at the end of every segment: Main consumes the last message of each register's chain and, unless the segment is the last one, re-emits the same value at the final memory step of the segment, where the next segment picks it up. A chain link therefore never spans more than one

segment, which is what keeps the 2^{24} bound sufficient. Register x_0 is not part of the chain: reads of x_0 are replaced by the immediate 0 and writes to it are dropped.

4.2.1.2. *Timestamps.*

The arithmetization does not inherently represent temporal order. On the other hand, the execution of instructions is inherently sequential. To encode it, ZisK discretizes time by attaching a *timestamp* to every memory access, so that the memory consistency argument can verify that reads always observe the most recent write to each address.

Time is organized at two granularities. A *main step* $t \in [0, 2^{36})$ indexes one full instruction execution by the Main. Within the j-th Main segment ($j = 0, 1, ...$; Section 5) the row $r \in [0, N)$ executes step

$$t = j \cdot N + r,$$

where the segment index j is an AIR value of the segment, range-checked to $[0, 2^{36}/N)$. The step is thus a globally monotone counter: the instruction at step t is executed before the instruction at step t' whenever $t < t'$.

A single main step, however, is not atomic from the memory's perspective: operand resolution requires reads, storing the result requires a write, and a dispatched precompile may independently access memory to read its inputs and write its outputs. To order all of these accesses, each main step t is subdivided into four *memory steps*

$$t_{\text{mem}} = 1 + 4 \cdot t + s, \quad s \in \{0, 1, 2, 3\},$$

with the following fixed slot assignment: $s = 0$ for the read of operand a, $s = 1$ for the read of operand b, $s = 2$ for the store of c and for the reads of a precompile dispatched at step t, and $s = 3$ for the writes of such a precompile and for the end-of-segment register reload of Section 4.2.1.1. Memory step 0 is reserved for the initial state, so that every access of the execution is strictly later than it. Since $t < 2^{36}$, memory steps fit in 38 bits.

These slots are the only ordering device: the address accessed at each slot is unconstrained and varies freely from instruction to instruction and from chip to chip. A precompile dispatched at step t receives t on the Operation Bus (Section 4.4.1) and stamps all of its memory accesses with slots 2 and 3 of that step —at addresses entirely different from those of the Main chip, and as many accesses as the operation needs— so the memory argument orders them exactly where the precompile's effect takes place in the execution. Sub-word stores, which the MemAlign chips implement as a read followed by a write of the enclosing word, stamp the write at the following memory step.

4.2.2. *Memory.*

The *Memory* chip centralizes all memory access verification. It is realised by three AIRs built from the same template, one per region of Table 2: Mem for the writable RAM, RomData for the ROM data and InputData for the input data. Each collects the aligned 8-byte operations submitted on the Memory Bus for its region, verifies their consistency, and rejects any execution in which a memory access violates the region's rules.

To enforce correctness, all operations of a region are sorted globally by address, then by memory step within the same address. This reduces the global consistency requirement —that every read observes the most recent prior write— to a sequence of local, row-to-row checks. Each row holds a word address addr (the byte address divided by 8), a memory step t_{mem}, a value (v_0, v_1) in the two-limb encoding of Section 4.3, a write flag wr, an address-change flag ρ and a selector sel indicating whether the row corresponds to an operation on the bus. Comparing each row with the previous one, the AIR enforces:

- *Address order*: if $\rho = 1$ the address strictly increases; otherwise it is unchanged.
- *Time order* (RAM only): within an address, the memory step strictly increases at every write and does not decrease at a read, so several reads of the same word may share a memory step. In the RAM AIR both orders are enforced by a single bounded increment: depending on ρ, either the address difference or the step difference (adjusted by $1 - \mathrm{wr}$), minus one, is decomposed into a 22-bit and a 16-bit range-checked limb. In the two read-only regions the value of an address never changes, so no time order is needed and only the address increment is range-checked.
- *No silent writes*: every write row has $\mathrm{sel} = 1$, i.e. corresponds to an operation on the Memory Bus, so no memory modification can occur outside the scope of the consistency argument.
- *Region policy*, depending on the AIR:
  - *RAM* (Mem): a read at an unchanged address returns the previous row's value, and a read at a new address returns zero, i.e. memory that has never been written reads as zero.
  - *ROM data* (RomData): the first access to each address is a write and all later accesses are reads returning the same value, so each word is written exactly once.
  - *Input data* (InputData): there are no writes; the value of each address is a prover-supplied constant shared by every access to it, and its four 16-bit chunks are range-checked. The only exception is the *free-input port*, the first word of the region, whose value may differ at every read: it is the channel through which hints are delivered (Section 2.3.9).
- *Region bounds*: the first address of the region's trace is at least the region's base address and the last one at most its end address.

Each address is therefore a contiguous block of rows, ordered in time in the RAM region, and the value continuity checks pin the value of every read to the most recent write of the same block. Formally, an *access* is a tuple $(\mathrm{wr}, \mathrm{addr}, t_{\mathrm{mem}}, v)$ of a write flag, a word address, a memory step and a value $v \in [0, 2^{64})$, and the rows of a region form a log in the following sense.

Definition 4.2 (Access Log). A *log* of a region is a finite sequence of accesses at word addresses of the region, each marked *submitted* or not, whose addresses do not decrease. For every access $(\mathrm{wr}, \mathrm{addr}, t_{\mathrm{mem}}, v)$, with predecessor $(\mathrm{wr}', \mathrm{addr}', t'_{\mathrm{mem}}, v')$ (none for the first access), the address is *new* if there is no predecessor or $\mathrm{addr} > \mathrm{addr}'$, and:

1. *RAM.* If the address is new, a read returns $v = 0$. Otherwise $t_{\mathrm{mem}} > t'_{\mathrm{mem}}$ for a write and $t_{\mathrm{mem}} \geq t'_{\mathrm{mem}}$ for a read, and a read returns $v = v'$. Every write is submitted.
2. *ROM data.* If the address is new, the access is a write if it is submitted and has $v = 0$ otherwise. Otherwise it is a read and returns $v = v'$. Memory steps are not constrained.
3. *Input data.* Every access is a read. If the address is not new and is not the free-input port, it returns $v = v'$. Memory steps are not constrained.

The Memory AIRs of a region hold one log of the region across all their segments, which are chained by a continuation bus (Section 5.3) so that the sorted order, and with it the checks above, holds across segment boundaries; the first address of the log is at least the base address of the region and the last one at most its end address. The submitted accesses are the aligned operations that the Memory AIRs prove on the Memory Bus, and the connection with the requesters is a *permutation* argument (Appendix A): the multiset of operations proved by the Memory AIRs must equal the multiset of operations submitted by the rest of the chips. The consequence that matters, *memory consistency* —every read of an execution returns the value of the last write before it at its

address, or the initial value of the region— follows from the log together with this permutation; the security analysis [Mas26] proves it as the offline memory check.

4.2.2.1. *Memory Access Paths.*

The access path for a memory operation depends on its width and alignment, as shown in Figure 2:

- **Aligned 8-byte access**: The requesting chip places the operation directly on the Memory Bus. The Memory chip processes it without further decomposition.
- **Sub-word or unaligned access** (1, 2, or 4 bytes): The requesting chip routes the operation to the *MemAlign* chip instead. MemAlign identifies whether the access falls within a single aligned 8-byte word or spans two consecutive words, decomposes it accordingly, and issues one or two aligned 8-byte operations to the Memory Bus.

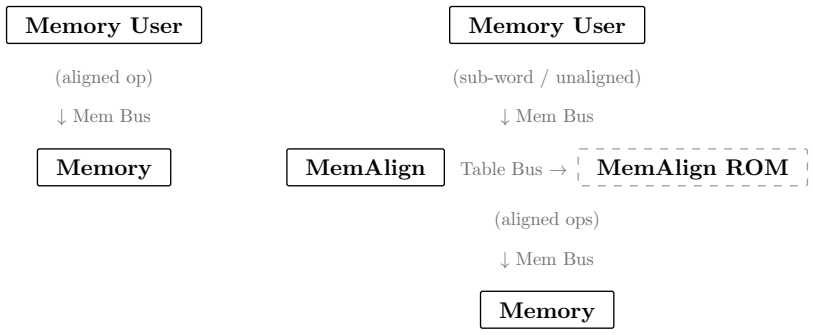


FIGURE 2. Memory access paths. Aligned 8-byte operations go directly to Memory; sub-word and unaligned operations are first decomposed by MemAlign into aligned 8-byte operations.

4.2.2.2. *MemAlign.*

The MemAlign chip models a small processor (the MemAlign ROM) with eight byte-wide registers that hold one aligned 8-byte word, and executes one of four fixed subprograms depending on whether the access is a read or write and whether it fits within one aligned word or spans two:

1. **Read within one word** [One read]: MemAlign performs one aligned read to load the surrounding 8-byte word into its registers, then extracts the requested bytes and returns the value.
2. **Write within one word** [One read, One write]: MemAlign reads the surrounding word, patches the relevant byte positions with the new value, writes the modified word back, and confirms the result.
3. **Read spanning two words** [Two reads]: The requested bytes straddle an 8-byte boundary. MemAlign reads both words, combines the relevant bytes from each, and returns the assembled value.
4. **Write spanning two words** [Two reads, Two writes]: MemAlign reads both words, patches the relevant bytes in each, and writes both modified words back.

In every case, MemAlign issues only aligned 8-byte operations to the Memory Bus. This design allows MemAlign to support all possible sub-word and unaligned accesses with a small, fixed set of constraints, without needing to know the specific access patterns in advance.

4.2.3. *Base Operations.*

The *Base Operations* chips prove the base instruction set operations —bitwise logic, comparisons, shifts, sign extensions, and integer multiplication and division— delegated from the Main chip via the Operation Bus.

These chips share a common structural profile that distinguishes them from the Main chip and from precompiles. First, they have no interaction with the Memory Bus: their inputs and outputs flow entirely through the Operation Bus, so they neither read from nor write to the memory state. Second, they carry no state across segment boundaries and therefore use no continuation bus; each segment's proof is self-contained. These two properties mean that Base Operations chips are the simplest kind of instruction executor in ZisK.

Base Operations chips together with the Main chip and Memory chip form the core execution engine of ZisK, responsible for executing every instruction in the ZisK ISA. These chips are not designed for any particular class of operations; rather, they are designed to be flexible enough to support the full range of operations in the ISA, and their constraints are structured to allow efficient delegation of complex operations to specialized sub-chips.

Precompiles, on the other hand, are designed to handle complex operations that would be expensive to constrain step-by-step in the Main chip.

4.2.4. *Precompiles.*

Precompiles handle classes of operations —elliptic curve arithmetic, cryptographic hash functions, bulk memory transfers— whose decomposition into base operations would be inefficient. Each precompile is an independent instruction executor chip that owns the full proof obligation for its operation.

What distinguishes precompiles from basic operation chips is the complexity of their interface with the rest of the system. A precompile typically reads its inputs and writes its outputs directly from the memory state through the Memory Bus, rather than receiving them as simple 64-bit operands on the Operation Bus. Its computation may span many internal rows —for example, a hash permutation iterates over multiple rounds— so its trace height is sized to the operation's intrinsic complexity, independently of the Main chip. Some precompiles (such as DMA) even carry their own internal state across segment boundaries via a continuation bus.

At the ISA level, invoking a precompile follows the calling convention of Section 2.3.10: the caller writes the operands to memory with ordinary instructions and then executes a single precompile instruction whose operand b points to them. The parameter-passing instructions are proved like any other instruction; the precompile instruction is dispatched by the Main chip on the Operation Bus together with its main step, and the precompile chip proves the whole memory effect of the operation, reading and writing through the Memory Bus at the memory steps of that main step (Section 4.2.1.2). Adding a new precompile requires only defining a new chip and wiring it to the appropriate buses, with no changes to any existing chip.

4.2.5. *Frequent-Operation Caches.*

Basic operation chips carry a fixed per-invocation overhead: each chip handles a whole family of operations, and a single query pays for the constraints that distinguish all of them, regardless of which one is actually being proved. Across a typical program execution, however, only a small fraction of all possible input combinations actually appear, and those that do appear tend to repeat. ZisK exploits this with *frequent-operation caches*: for binary and arithmetic operations, results for the most commonly queried input pairs are pre-computed and committed once at setup time.

At proof time, sub-chips resolve queries directly from the cache for inputs it covers, and fall back to constraint checking for the remainder. This significantly reduces witness size and proving time for programs with typical workloads.

4.3. **Word Encoding over Goldilocks.**

The ZisK ISA operates on 64-bit words, i.e. integers in $[0, 2^{64})$ with arithmetic modulo 2^{64}, while the arithmetization works over the Goldilocks field, whose modulus $p = 2^{64} - 2^{32} + 1$ is smaller

than 2^{64}. A word therefore does not fit in a single field element, and integer overflow must happen at 2^{64}, not at p. ZisK bridges this gap with a uniform encoding that every chip and every bus follows.

- **Two 32-bit limbs.** A word v is represented by two field elements (v_0, v_1) with $v = v_0 + 2^{32} \cdot v_1$ and $v_0, v_1 \in [0, 2^{32})$. Since $2^{32} < p$, every limb has a unique representative in $\mathbb{F}$, and the pair determines v. The Main chip's operand and result columns, the Operation Bus, the Memory Bus and the Memory AIRs all use this form (Section 4.4); the 32-bit opcodes simply set the high limb of their inputs to zero.
- **Range checks at the producer.** A limb is meaningful only if it lies in $[0, 2^{32})$. ZisK enforces this where a value is *produced*: the executor and precompile chips decompose their results into smaller pieces —bytes in Binary and BinaryExtension, 16-bit chunks in BinaryAdd, Arith and the 256- and 384-bit precompiles— which are range-checked through the table buses of Section 4.4.4, and rebuild each limb from them, e.g. $v_0 = \sum_{k=0}^{3} 2^{8k} \beta_k$ from bytes $\beta_k \in [0, 2^8)$. Likewise, values entering from the input data region are range-checked by the InputData AIR in 16-bit chunks. Chips that only *move* a value —Main forwarding an operand, the Memory AIRs storing it— do not check it again: the bus transports the exact limbs, so a consumed value inherits the range established by its producer.
- **Overflow at 2^{64}.** Arithmetic is verified on the decomposed form, with explicit carries. For instance, BinaryAdd proves $c = a + b \bmod 2^{64}$ through

$$a_0 + b_0 = c_0 + 2^{32}\kappa_0, \quad a_1 + b_1 + \kappa_0 = c_1 + 2^{32}\kappa_1, \quad \kappa_0, \kappa_1 \in \{0, 1\},$$

  discarding the final carry κ_1, and Arith proves multiplications and divisions as integer identities over 16-bit chunks with range-checked carries. Every term of these identities is bounded well below p by the range checks, so none of them can wrap around modulo p: they hold over the integers, and hence modulo 2^{64} as the ISA requires.
- **Addresses and steps.** Byte addresses (32 bits), program counters (32 bits) and memory steps (38 bits) are below p and are carried as single field elements, bounded by the range checks of the chips that use them (Section 4.2.2, Section 4.2.1.2).

4.4. System Buses.

ZisK uses a bus system —recall Table 3 for an overview— to constrain valid state transitions during program execution. Now we discuss all of them except the continuation bus, which is described in Section 5.3.

Every bus is an instance of the bus-balancing primitive of Section 3, identified by a fixed bus identifier and carrying messages of a fixed schema. All buses are reduced by the same LogUp argument (Section 3.3.1) under the global challenges of Section 6.2; they differ in the argument they realise (Appendix A). Values in $[0, 2^{64})$ travel as two 32-bit limbs, following Section 4.3.

4.4.1. *Operation Bus.*

The *operation bus* routes complex operations from the Main chip to specialized sub-chips. Messages have the form:

$$(\text{op}, a_0, a_1, b_0, b_1, c_0, c_1, \text{flag}, t, x, \text{extra}),$$

where $\text{op} \in [0, 2^8)$ identifies the operation (Appendix B), (a_0, a_1) and (b_0, b_1) are the input operands, (c_0, c_1) is the output, $\text{flag} \in \{0, 1\}$ is the operation's flag, t is the main step (Section 4.2.1.2), x is the *extended argument* of a precompile instruction (Section 2.3.2), and extra carries additional information used only between the DMA chips.

The main step t plays two different roles depending on the kind of sub-chip on the receiving end. For sub-chips whose entire input/output is already conveyed by (a, b, c) —i.e. those that do not interact with memory— t is set to 0 and carries no information: the operation is fully specified by its operands. For precompiles, by contrast, (a, b, c) is not enough: precompiles read their inputs and write their outputs directly through the memory bus rather than through the operand triple, so each precompile invocation must be tied to a specific point in the execution timeline. In that case, t is set to the current execution step of the Main chip, allowing the precompile's memory accesses to be ordered consistently with the rest of the trace.

When the Main chip delegates an operation, it sends a message on the operation bus and asserts that $(c, \text{flag}) = \text{op}(a, b)$ without verifying the operation locally; for the 32-bit opcodes it zeroes the high limbs a_1, b_1. The executor or precompile chip in charge of the opcode (Table 4) receives those messages and proves correctness through its own constraints, as do the frequent-operation caches for the inputs they cover. The bus is a lookup argument: each receiving chip proves a message with a free multiplicity, and executor chips cancel the contribution of their unused (padding) rows with a single term per segment, derived from its AIR values.

4.4.2. *ROM Bus.*

The *ROM bus* ensures correctness of instruction fetching. Messages are instructions in the format of Section 2.3.2, keyed by their address:

$$(\text{pc}, \text{a_off}, \text{a_imm1}, \text{b_off}, \text{b_imm1}, \text{ind_w}, \text{op}, \text{store_off}, \text{j1}, \text{j2}, \text{flags}),$$

where $\text{flags} = 1 + \sum_k 2^k f_k$ packs the fifteen boolean fields f_k of the instruction (source and store kinds, `store_pc`, `set_pc`, `m32`, `is_external_op` and `is_precompiled`). The constant 1 makes every instruction distinct from the all-zero padding rows of the table.

The Main chip sends one message per row, built from the instruction columns of that row; the Rom chip proves the program as a lookup table, with one row per instruction and a free multiplicity column. The Rom chip's table columns are *cached* (Section 3): they are committed once per program, and their Merkle root is the *program commitment* root_{ROM}, one of the publics of every ZisK proof (Section 6.3). Balancing of the ROM bus therefore implies that the instruction executed at every Main row is the one stored at its program counter in the committed program.

4.4.3. *Memory Bus.*

The *memory bus* is the channel through which every chip that reads from or writes to an address space submits its memory operations. Messages have the form:

$$(\text{op}_{\text{mem}}, \text{addr}, t_{\text{mem}}, \text{width}, v_0, v_1),$$

where $\text{op}_{\text{mem}} \in \{\texttt{LOAD} = 1, \texttt{STORE} = 2, \texttt{REG} = 3\}$ is the access type, $\text{addr} \in [0, 2^{32})$ is the target byte address (the register index for `REG` messages, Section 4.2.1.1), t_{mem} is the memory step (Section 4.2.1.2), $\text{width} \in \{1, 2, 4, 8\}$ is the access width in bytes, and (v_0, v_1) is the value read or written.

The requesters —Main, the MemAlign chips and the precompiles— submit their operations; the three Memory AIRs prove the aligned 8-byte ones; the MemAlign chips prove the sub-word and unaligned ones and in turn submit the aligned operations they decompose them into (Figure 2); and Main both submits and proves the `REG` messages of the register argument. The bus is a permutation argument, so balancing it, together with the constraints of the Memory AIRs, implies that every read observes the most recent write to its address.

4.4.4. *Table Buses.*

The remaining buses connect executor and precompile chips to fixed tables, which are packed into the SpecifiedRanges, VirtualTable0 and VirtualTable1 AIRs (Table 4). Each table bus is a

lookup argument whose receiving side is a fixed table with a free multiplicity column, so balancing implies that every message sent on it is a row of the table. Table 5 lists them.

Table	**Rows**	**Queried by**
Binary table	one byte of a byte-sliced comparison, subtraction or bitwise operation, with carry in and out	Binary
Binary extension table	one byte of a shift or sign extension	BinaryExtension
Arith table	decoding of the Arith operations and of their sign and range selectors	Arith
Arith range table	16-bit chunks, sign-dependent chunk ranges and carry ranges of Arith	Arith
MemAlign program	the micro-program executed by MemAlign	MemAlign
ArithEq comparison table	chunk-wise comparison used for modular reduction	ArithEq, ArithEq384
Keccak-f table	the bit-sliced Keccak-f round functions	Keccakf
DMA tables	DMA micro-program, partial-word patterns and byte comparisons	DMA chips
Dual range tables	pairs of values in $[0, 2^7)$, resp. $[0, 2^8)$	MemAlign, DMA chips
Range checks	one bus per range $[x, y]$ used by some chip	all chips

TABLE 5. Table buses of ZisK. The frequent-operation caches of Section 4.2.5 are also fixed tables, but they receive on the Operation Bus instead of a bus of their own.

Finally, a few buses are internal to a single chip: the DMA chips use a permutation bus to hand a bulk operation from the front-end AIR to its head-and-tail AIR, and Blake2br uses a permutation bus to apply its message permutation. Neither carries messages from outside the chip.

4.4.5. *Meaning of the Buses.*

Balancing a bus states only that its messages are sent and received with equal total multiplicities. What a message *means* is a contract between the chips on the bus. A chip *proves* a message when it adds it to the bus as a provider —the receiving side of a lookup, or the side of a permutation that holds the data— and *assumes* it when it relies on it, as the Main chip relies on the results of the operations it delegates. Every chip that proves a message on a bus must establish the contract of that bus by its own constraints, and every chip that assumes a message may take the contract for granted: balancing pairs every assumed message with a proved one. The contracts of ZisK are the following.

- **Operation Bus.** A message $(\mathrm{op}, a_0, a_1, b_0, b_1, c_0, c_1, \mathrm{flag}, t, x, \mathrm{extra})$ states that, for $a = a_0 + 2^{32}a_1$, $b = b_0 + 2^{32}b_1$ and $c = c_0 + 2^{32}c_1$, $(c, \mathrm{flag}) = \mathrm{op}(a, b)$ as specified in Appendix B, and that $c_0, c_1 \in [0, 2^{32})$ whenever $a_0, a_1, b_0, b_1 \in [0, 2^{32})$ (Section 4.3). For a precompile it states moreover that the chip makes on the Memory Bus, at the memory steps $1 + 4t + 2$ and $1 + 4t + 3$, exactly the reads and writes of the operation's memory effect with extended argument x. For `pubout` the contract is instead that b is the public output of index a: its only prover is the global message of the public statement (Section 6.3).
- **ROM Bus.** A message $(\mathrm{pc}, ..., \mathrm{flags})$ states that the instruction of the committed program at address pc has the fields of the message.
- **Table buses.** A message is a row of its table; on a range bus, the value lies in the range.
- **Memory Bus and continuation buses.** These carry no contract. Their meaning comes from counting: balancing states that the multiset of messages assumed equals the multiset

of messages proved, and the access log (Definition 4.2) and the boundary equations (Section 5.3) turn this equality into memory consistency and into the chaining of segments. The contracts are relied on without cycles, at the level of operations. The Main chip assumes operation results and ROM entries. On the Operation Bus, two chips also assume operations, of kinds they do not prove: Arith assumes the comparisons it delegates to Binary, and the DMA front-ends assume the internal operations of the DMA workers; the chips that prove these internal operations assume no operation. The operation and precompile chips further assume table rows and memory accesses, and the tables, the Rom chip and the global messages assume nothing. Since no proof of a message relies, even indirectly, on a message of its own kind, balancing transfers every contract from the chips that prove it to the chips that assume it.

5. Continuations

A single chip proof covers a trace of fixed height N. Real programs run for an a priori unbounded number of steps, so a single proof cannot certify an arbitrary execution. *Continuations* are the mechanism by which a zkVM extends beyond this bound: the execution is divided into *segments*, each covered by an independent proof, and the proofs are linked into a single argument for the full run.

5.1. The Standard Approach to Continuations.

The most common continuation strategy splits execution into *checkpointed* segments. Execution runs for N steps, halts, and the prover generates a proof for those N steps; the machine state at the halt —the *checkpoint*— is saved. A fresh execution then resumes from that checkpoint and runs for another N steps, and so on. Each segment proof certifies one N-step block, and consecutive proofs are chained by having each segment verify the preceding checkpoint as part of its own public inputs.

This approach carries a significant overhead: *padding*. If a chip performs only $k \ll N$ operations in a given segment, its $N - k$ remaining rows must be filled with dummy entries that satisfy the constraints but carry no useful computation. In a multi-chip system, this cost compounds across every chip in every segment: different chips have wildly different utilization across segments, yet each padded row still enters the polynomial commitment and constraint evaluation. The result is a large blowup in prover time and proof size, which grows with the number of segments and the number of chips.

A solution at the cryptographic layer is the *jagged polynomial commitment scheme (Jagged PCS)* [Hem+25]. This protocol commits to the full computation trace as a single polynomial over a jagged matrix —where each column corresponds to a chip and can have a different height— and lets the verifier emulate access to the individual per-chip polynomials. The prover pays proving cost proportional to the actual trace area rather than the padded one. The jagged PCS is, however, inherently multilinear: it is built on multilinear polynomial commitments and can be incorporated efficiently only into proving systems whose arithmetization is multilinear-based.

Rather than developing new cryptographic infrastructure and tying the solution to a specific class of proving system, ZisK addresses the problem at the system design level, described next.

5.2. ZisK's Approach: Trace Splitting.

ZisK decouples segmentation from execution and lets chips operate on demand. The program runs to completion in the host execution environment in a single pass, producing the full execution trace. Each chip records only the rows for instructions it handles: the Memory chip records only memory accesses, Keccak only hash operations, and so on. Chips that are rarely invoked produce short traces; chips that are heavily used produce long ones.

The prover then partitions each chip's trace independently into consecutive segments of that chip's fixed height N_i. Padding is limited to the final segment of each chip, where the last partial slice is padded to N_i rows; all earlier segments are filled to capacity. This stands in contrast to the standard approach, where every chip is padded to its height in every execution segment regardless of how many operations it actually performed. Figure 3 illustrates this layout.

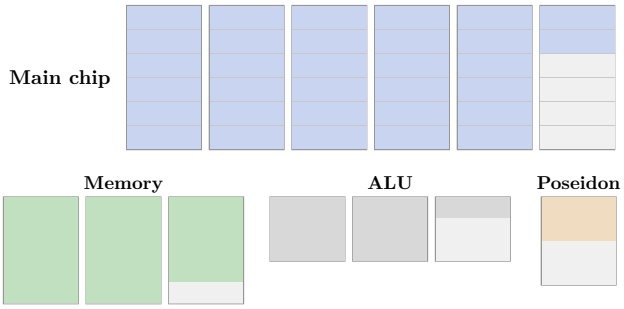


FIGURE 3. Padding layout for ZisK's trace-splitting approach. Chips define their own segment height N_i. Colored rows hold actual computation; gray rows are padding. Only the final segment of each chip requires padding.

We now make the segmentation precise. Fix an execution $S_0, ..., S_T$ of a program (Section 2.4). Every step t contributes a deterministic sequence of rows to some of the AIRs of ZisK (Table 4): one row to Main, one row to the chip in charge of its opcode if the operation is delegated, as many rows as the operation needs to a precompile, and one row per memory access to the Memory AIR of the accessed region (possibly through MemAlign). For each AIR $\mathcal{C}_i$, the concatenation of these rows over all steps, sorted by address and memory step for the Memory AIRs, is the *global trace* $\boldsymbol{T}_i^{\text{glob}}$ of $\mathcal{C}_i$.

Definition 5.1 (Segmentation). Let N_i be the trace height of AIR $\mathcal{C}_i$. The *segmentation* of an execution splits every global trace into consecutive slices of N_i rows,

$$\boldsymbol{T}_i^{\text{glob}} = \boldsymbol{T}_{i,0} \parallel \boldsymbol{T}_{i,1} \parallel ... \parallel \boldsymbol{T}_{i,m_i-1}, \quad m_i = \left\lceil \left| \boldsymbol{T}_i^{\text{glob}} \right| / N_i \right\rceil,$$

where only the last slice is padded to N_i rows ($m_i = 0$ if $\boldsymbol{T}_i^{\text{glob}}$ is empty). A *segment* is one such slice $\boldsymbol{T}_{i,j}$, together with its AIR values $\boldsymbol{a}_{i,j}$; it is a witness for $\mathcal{C}_i$ (Section 3), proved on its own. Its AIR values include the segment index j, a flag indicating whether $j = m_i - 1$, and the *boundary payloads* $B_{i,j}^{\text{in}}$ and $B_{i,j}^{\text{out}}$ that summarise the state of the chip at the start and at the end of the segment (Section 5.3).

For the Main chip, segment j covers exactly the steps $t \in [jN, (j+1)N)$, with $N = N_{\text{Main}}$: as recalled in Section 4.2.1.2, the AIR reconstructs the step of each row from the segment index j, so the assignment of steps to Main segments is enforced by the constraints. For every other chip, the partition is by row count and does not align with step boundaries. This is why cross-segment consistency in ZisK is not an equality of whole VM states between consecutive segments, as in the standard approach, but decomposes into per-chip boundary payloads (Section 5.3).

Trace splitting eliminates most padding —in fact, it is equivalent to the Jagged PCS approach — but it introduces two problems that must be solved.

The first is *cross-chip consistency*. As commented in Section 4, there are chips that delegate operations to other specialized chips and record the expected result. Proving that the delegation was correct requires that the involved chips agree on the exchanged values. In the standard continuation's solution this agreement is enforced directly by generating a single monolithic proof

of the entire trace. In the trace-splitting design the proofs for different chips are generated independently. Binding them into a globally consistent argument requires a shared challenge that spans all chip instances —a check that cannot be performed locally and is instead delegated to the recursive aggregation layer. We explain the solution to this problem in Section 6.

The second is *cross-segment state*. Because each chip's trace is divided independently, a chip whose state evolves over the course of execution cannot restart cold at the beginning of each segment: consecutive segment instances must agree on the machine state at their shared boundary. This is the problem that continuations, in the classical sense, solve.

For chips whose computation is self-contained within each segment —such as base operations — the cross-segment state is empty: consecutive segment instances are independent and carry no shared context. For chips with persistent state, however, consecutive instances must agree on the state at their shared boundary. This shared boundary state is enforced by the *continuation bus*.

5.3. The Continuation Bus.

The *continuation bus* carries cross-segment state for each chip that requires it. Each segment instance receives, as a message on the bus, the boundary payload B^{in} it starts from, and sends the boundary payload B^{out} it ends with. The AIR constraints tie B^{in} to the first row of the segment and B^{out} to its last row. Bus balancing (Section 3.4) ensures the chain is globally consistent: every sent message is received exactly once with matching content, so gaps, duplications, and reorderings are all ruled out. Two *anchors* —global terms contributed once per proof rather than by any segment — open and close each chain: the *genesis* anchor sends the payload the first segment must start from, and the *terminus* anchor receives the payload the last segment must end with. Continuation messages are built from AIR values, so each segment contributes exactly one received and at most one sent message per continuation bus.

The payload is chip-specific, and every chip with state that must survive a segment boundary defines its own continuation bus. Every message carries the segment index, so that the chain can only be assembled in order: the message sent by segment j carries index $j+1$ and can only be received by the segment with index $j+1$.

Main. The payload is the control state (pc, c) of the guest (Section 2.4): the program counter of the next instruction and the output of the last one. Messages have the form $(j, \text{last}, \text{pc}, c_0, c_1)$. Segment j receives $\big(j, 0, \text{pc}_j^{\text{in}}, c_j^{\text{in}}\big)$, where pc_j^{in} is the program counter of its first row and c_j^{in} is the value that its first row reads through the source `C`. It sends $\Big((j+1)\big(1-\text{last}_j\big), \text{last}_j, \text{pc}_j^{\text{out}}, c_j^{\text{out}}\Big)$, where pc_j^{out} is the next program counter computed by its last row and c_j^{out} is the output of its last row, and where last_j is the last-segment flag. The genesis anchor sends $(0, 0, \text{pc}_{\text{BOOT}}, 0, 0)$ and the terminus anchor receives $(0, 1, \text{pc}_{\text{END}}, 0, 0)$. With m Main segments, balancing therefore enforces the boundary equations

$$(\text{pc}_0^{\text{in}}, c_0^{\text{in}}) = (\text{pc}_{\text{BOOT}}, 0), \quad \Big(\text{pc}_j^{\text{out}}, c_j^{\text{out}}\Big) = \big(\text{pc}_{j+1}^{\text{in}}, c_{j+1}^{\text{in}}\big) \text{ for } j < m-1, \quad (\text{pc}_{m-1}^{\text{out}}, c_{m-1}^{\text{out}}) = (\text{pc}_{\text{END}}, 0).$$

Since segment j covers the steps $[jN, (j+1)N)$, the middle equation states that the control state at step $(j+1)N$ is the same whether it is computed by segment j or assumed by segment $j+1$, and the outer ones are the initial and terminal conditions of Section 2.4. Setting the index of the message sent by the last segment to 0 makes the terminus the only possible receiver of the final handoff, so the last segment's output cannot be re-consumed as the input of a further segment.

The rest of the guest state needs no payload of its own. The registers are carried across the boundary by the register argument of Section 4.2.1.1, which re-emits every register at the last memory step of each segment. The memory contents are never carried across a boundary: they are determined by the sorted access trace of each memory region, which spans the whole execution.

Memory AIRs. The payload is the last row $(\text{addr}, t_{\text{mem}}, v_0, v_1)$ of the segment's slice of the sorted access trace (Section 4.2.2), so that the order and value checks continue across the boundary. Messages have the form $(\text{base}, j, \text{addr}, t_{\text{mem}}, v_0, v_1)$, where the region's base address separates the chains of the three regions. Segment j receives $\left(\text{base}, j, B_j^{\text{in}}\right)$, which plays the role of the row preceding its first row, and, unless it is the last segment, sends $\left(\text{base}, j+1, B_j^{\text{out}}\right)$ with B_j^{out} its last row. The genesis anchor sends $(\text{base}, 0, \text{base}\ /8, 0, 0, 0)$: the first row of the region is compared against a fictitious access at the base address, at memory step 0, with value 0. There is no terminus: the last segment simply sends nothing, and its last address is checked against the end of the region instead. The boundary equations are

$$B_0^{\text{in}} = (\text{base}\ /8, 0, 0, 0), \quad B_j^{\text{out}} = B_{j+1}^{\text{in}} \text{ for } j < m-1.$$

The genesis anchor of the RAM region is unconditional; those of the ROM data and input data regions are switched on by proof-level boolean flags, so that a program that never touches one of these regions needs no segment of its AIR.

DMA. The aligned and unaligned DMA workers (Table 4) may split a single bulk operation across a segment boundary. Their payload is the operation in flight —source and destination addresses, remaining byte count, main step, and for unaligned copies the bytes read ahead— together with a flag indicating whether an operation is in flight at all; their genesis anchors send the payload "no operation in flight". The messages follow the same pattern as above, on a continuation bus of their own.

Other chips. The remaining chips —the base-operation chips, the hash and arithmetic precompiles, the MemAlign chips and the DMA front-ends— have no cross-segment state and use no continuation bus.



FIGURE 4. Continuation bus for the Main chip. The genesis anchor injects the boot program counter; the terminus anchor receives the final handoff at the designated end program counter.

5.4. Parallel Proving and Recursive Aggregation.

Because each segment instance is proved independently, all segments across all chips can be proved in parallel. Each individual proof establishes only *local consistency*: it certifies that the segment's own constraints are satisfied and records its contribution to the shared bus interactions as a partial sum. Two global checks remain outstanding.

First, the continuation buses must balance globally. Every cross-segment state message sent by a segment instance must be received exactly once by the next, with matching content. No individual segment proof can verify this —it requires aggregating the partial sums from all instances of a chip across all of its segments.

Second, the cross-chip buses must balance globally. The partial sums recording interactions between the Main chip and the Memory, or other specialized chips must cancel to zero when combined. This is the cross-chip consistency problem introduced earlier, and it likewise cannot be enforced locally.

Both checks reduce to a single global field equation: the sum of all partial sums across all instances and all chips must equal zero. The recursive aggregation layer (Section 6) accumulates these partial sums up an aggregation tree and enforces this equation at the root, binding the independent segment proofs into a single, globally valid argument for the full unbounded execution.

6. Recursion

ZisK uses proof recursion to combine the independent per-segment proofs produced by trace splitting into a single, globally consistent argument. This section introduces proof recursion in general, identifies the two specific problems it resolves in ZisK, and details the aggregation structure.

6.1. **Proof Recursion.**

A proving system is called *recursive* when it can verify the validity of another proof as part of its own computation. The key observation is that the verifier V is itself a computation —it reads a proof, the verification key, and the public statement, and performs arithmetic checks— so it can be expressed as polynomial constraints and be proved. A *recursive verifier* is therefore a computation expressed as constraints —in ZisK, an AIR— that takes a proof π as witness and a public statement x as input, and checks that $\mathbf{V}(\mathsf{vk}, x, \pi)$ accepts for some verification key vk. Proving it yields a *recursive proof*: a succinct argument that attests to the correctness of an underlying proof without re-exposing its content.

The power of this construction comes from its composability. Since a recursive proof is itself a valid proof, it can in turn be fed into another recursive verifier, producing a proof of a proof of a proof. Iterating this idea allows a sequence of computations —or a collection of independent computations— to be folded into a single, constant-size argument regardless of the number of steps or the size of the individual proofs. The result is a *proof composition* that scales arbitrarily: adding more computations does not make the final proof larger or more expensive to verify.

Recursive composition is normally used for three purposes. The first is *proof compression*: reducing the size of a proof by recursively re-proving its verifier within the same proving system. The second is *proof aggregation*: combining many proofs $\pi_1, ..., \pi_M$ into a single argument that attests to the validity of all of them simultaneously. The third is *cross-system conversion*: translating a proof generated with one proving system (e.g., a proof with large size but fast to generate) into a proof in another system better suited for a particular context (e.g., a compact proof with fast verification).

ZisK leverages recursion for the previous purposes —trace splitting (Section 5) produces many segment proofs that need to be aggregated; intermediate aggregation proofs are themselves recursively re-proved to shrink their size; and the topmost proof can be converted into a format suitable for the external verifier— but it also relies on it to resolve two open problems left by trace splitting itself.

The first is *cross-chip bus balancing*. Each segment proof contributes a partial sum $s_i \in \mathbb{K}$ to the global bus interaction. Global correctness requires $\sum_i s_i = 0$, but no individual proof can verify this: it requires aggregating the partial sums across all segment instances and all chips.

The second, and more subtle, is *challenge derivation*. The LogUp argument (Section 3) uses random field elements α and γ, derived via the Fiat-Shamir heuristic, to compress bus interactions into a single accumulated value per AIR. For the global balancing equation $\sum_i s_i = 0$ to be meaningful, all partial sums must be computed under the *same* challenges. In a monolithic proof this is automatic: there is one prover transcript and one application of Fiat-Shamir. In the trace-splitting design, each segment is proved independently and would naturally derive its own challenges from its own commitments —producing values that differ across instances and making the partial sums incommensurable.

6.2. **Challenge Derivation.**

The challenge derivation problem arises because Fiat-Shamir ties each challenge to the transcript of a single prover. When segment provers operate independently, their transcripts diverge and so do their challenges.

The solution is to separate challenge derivation from proof generation: all segment provers first commit to their traces, and the challenge is then derived from the full set of commitments and broadcast to all provers as a *public input*. Each segment proof proves the statement "given the key $\chi = (\alpha, \gamma)$, my witness satisfies my constraints and my partial sum is s_i" —the key is *assumed*, not derived internally.

A natural first attempt is plain hashing: Given commitments $\mathrm{com}_1, ..., \mathrm{com}_M$, compute:

$$\chi = H(\cdots H(H(\mathrm{com}_1, \mathrm{com}_2), \mathrm{com}_3)\cdots, \mathrm{com}_M).$$

This fails for two reasons. It is inherently sequential, which negates the parallelism of trace splitting; and it is order-dependent: changing the processing order changes the challenge, so partial batches cannot be computed independently and merged.

ZisK instead uses a *multiset hash* [Cla+03] —a function $\mathcal{H}$ from multisets over a message space $\mathcal{M}$ to a commutative group $(\mathcal{G}, +)$, satisfying two properties:

- *Multiset Homomorphism*: $\mathcal{H}(S \cup T) = \mathcal{H}(S) + \mathcal{H}(T)$ for any two multisets S, T over $\mathcal{M}$.
- *Multiset Collision-Resistance*: finding distinct multisets $S \neq T$ with $\mathcal{H}(S) = \mathcal{H}(T)$ is computationally infeasible.

The standard construction builds $\mathcal{H}$ from an inner *hash-to-group* function $h : \mathcal{M} \to \mathcal{G}$ as

$$\mathcal{H}(S) = \sum_{x \in S} m_S(x) \cdot h(x),$$

where $m_S(x) \in \mathbb{N}$ is the multiplicity of x in S. As a consequence, segment provers can independently hash and sum their own commitments, and partial accumulators from disjoint subsets can be merged later by addition. The common key is then obtained from the global accumulator $\mathrm{acc} = \mathcal{H}(\{\mathrm{com}_1, ..., \mathrm{com}_M\})$ by a deterministic *key derivation*. It also absorbs the public statement $\boldsymbol{p}$ (Section 6.3), so that the key is bound to the statement as well as to the commitments:

$$\chi = (\alpha, \gamma) = \mathsf{DeriveKey}(\boldsymbol{p}, \mathrm{acc}) \in \mathbb{K}^2.$$

ZisK instantiates DeriveKey with Poseidon2: a sponge absorbs $\boldsymbol{p}$ and acc and squeezes a *global challenge* in $\mathbb{K}$, from which a fresh sponge squeezes α and γ. Multiset collision-resistance ensures that the only way for a malicious prover to alter χ is to alter the statement or the multiset of commitments, which would in turn invalidate at least one segment proof.

ZisK instantiates this scheme with *LtHash* [Zis25], a lattice-based multiset hash. The group $\mathcal{G}$ is the vector space $\mathbb{Z}_p^n$. The element hashed for a segment is its *typed commitment* (τ, com): the type τ of its AIR, identified by the root of the AIR's fixed columns, together with com, the Merkle root of its stage-1 trace commitment (Section 7) and its stage-1 AIR values. The inner hash h absorbs (τ, com) into a Poseidon2 sponge [GKS23] and squeezes its 16-element state, then applies 22 further Poseidon2 permutations in a chain, concatenating the 23 states into a pseudorandom vector $h(\tau, \mathrm{com}) \in \mathbb{Z}_p^n$ with $n = 23 \cdot 16 = 368$. Multiset collision-resistance for LtHash reduces to the Short Integer Solution (SIS) [Ajt96] lattice problem, which is believed to remain hard even against quantum adversaries.

The LtHash group has finite exponent p: adding p copies of the same element to a multiset leaves $\mathcal{H}$ unchanged, since $p \cdot h(x) \equiv 0 \in \mathbb{Z}_p^n$. To rule out such trivial collisions, the scheme is restricted to multisets of cardinality less than $2^B \ll p$, with $B = 20$. ZisK enforces this bound by tracking the multiset cardinality alongside the accumulator, via the leaf-count counter κ that the recursion carries and range-checks against 2^B at every aggregation node (introduced in the next section).

Equivalently, one may think of LtHash as using an augmented per-element hash $h'(x) = (h(x), 1) \in \mathbb{Z}_p^{n+1}$, so that the last coordinate of the multiset hash tracks the total cardinality automatically.

Correctness of the assumed key χ must be verified —otherwise a malicious prover could substitute any χ to make the partial sums balance artificially. Since $\mathcal{H}$ is homomorphic over multiset union, this verification is embedded directly into the aggregation tree: each node carries a *partial accumulator* over the commitments in its subtree, and a parent node combines the two by addition. At the root, the fully accumulated value is checked against the claimed χ, confirming that it was correctly derived from the statement and the entire set of commitments.

6.3. **What ZisK Proves.**

Before describing the recursive verifiers, we make explicit what ZisK as a whole proves. Given a fixed program P, ZisK proves that there exists a successful execution of P (Section 2.4), on some input and some hint stream, that declares a given list of public outputs: formally, a witness of the execution relation $\mathcal{R}_{\text{exec}}$ of Definition 2.2. The data exposed to the verifier form the *external statement* $\boldsymbol{p}_{\text{ext}} \in \mathbb{F}^{68}$:

- the program commitment $\text{root}_{\text{ROM}} \in \mathbb{F}^4$, the Merkle root of the Rom chip's cached columns, which hold the program code (Section 4.4.2);
- the 32 public outputs $\text{out}_0, ..., \text{out}_{31} \in [0, 2^{64})$ of Section 2.3.8, each as two 32-bit limbs (Section 4.3), for 64 field elements.

Everything else —the input, the hints, the intermediate states of the execution— stays inside the prover. A proof additionally carries 8 *proof values* $\boldsymbol{p}_{\text{pv}} \in \mathbb{F}^8$: boolean flags, chosen by the prover, that switch on the genesis anchors of the optional AIRs (the ROM data and input data regions and the DMA variants, Section 5.3). They are not part of the external statement, but every node of the recursion carries them together with it, so we call the pair $\boldsymbol{p} = (\boldsymbol{p}_{\text{ext}}, \boldsymbol{p}_{\text{pv}})$ the *public statement* of the tree, following the security analysis [Mas26].

These publics are bound to the execution by two mechanisms. The program commitment is the root of a commitment that the Rom AIR's proof opens, so every instruction fetched on the ROM Bus belongs to the program committed by root_{ROM} (Section 4.4.2). The public outputs are bound through *global messages*: messages that no AIR emits, but that the system as a whole adds to the buses as a function of $\boldsymbol{p}$. We write $\mathcal{G}(\boldsymbol{p})$ for the multiset of global messages of $\boldsymbol{p}$. It contains

- the 32 messages $(\texttt{pubout}, i, 0, \text{out}_{i,0}, \text{out}_{i,1}, \text{out}_{i,0}, \text{out}_{i,1})$ received on the Operation Bus for $i \in [0, 32)$, where $(\text{out}_{i,0}, \text{out}_{i,1})$ are the limbs of out_i. The only sender of `pubout` messages is the Main chip, at the `pubout` instructions of the epilogue, so balancing the Operation Bus forces the execution to declare exactly the outputs in $\boldsymbol{p}_{\text{ext}}$;
- the genesis and terminus anchors of the continuation buses (Section 5.3), the optional ones gated by $\boldsymbol{p}_{\text{pv}}$;
- the initialisation of the registers on the Memory Bus (Section 4.2.1.1).

The execution is split into segments and proved piecewise (Definition 5.1): given the family of chips $\mathfrak{A} = \{(\mathcal{C}_\tau, \mathcal{I}_\tau)\}_{\tau \in [M]}$ (Table 4), each segment is a witness for one of the AIRs $\mathcal{C}_\tau$. The segments are stitched together by global bus balancing —which includes the continuation buses of Section 5.3 and the global messages $\mathcal{G}(\boldsymbol{p})$— under the key χ derived from the statement and the prover's commitments as described in Section 6.2.

6.4. **Recursive Structure.**

ZisK assembles its proof as the aggregation tree illustrated in Figure 5. The leaves of the tree are the individual AIR segment instances; the internal nodes are *recursive verifiers* —AIRs whose constraints verify their children's proofs and expose a small number of public values that propagate upward. Internal nodes have a fixed arity, but the tree is not required to be balanced: it is built

opportunistically as proofs complete, so a branch that finishes early can be verified immediately while others are still running. On top of the aggregation tree sits a single *Final Verifier*, which is the only node that performs the global checks —bus balancing across all segments, and consistency of the key χ.

The key χ is an input to the entire structure, not something derived inside it. Before proving starts, χ is derived according to Section 6.2 and broadcast to every leaf prover. Each leaf proof takes χ as a public input and uses it to compute its bus contribution. The tree then recomputes, as it aggregates, the quantities from which χ was derived, and the Final Verifier checks that the key the leaves used is the one derived from what was actually aggregated.

The tree has four kinds of nodes: AIR leaves, leaf verifiers, aggregation verifiers, and the Final Verifier. For some AIRs a fifth kind, the *compressor*, sits between the AIR leaf and its leaf verifier: the STARK proofs of these AIRs are large, and verifying them directly would make the leaf verifier large as well. A compressor is a recursive verifier that verifies the AIR proof and re-proves the same statement at a lower rate (Section 7), so that the leaf verifier verifies the compressor's proof instead. The precise relation that every kind of node attests to is stated, with the analysis of what the relations establish together, in the companion security analysis [Mas26]; here we describe what each node checks.

6.4.1. *Accumulators.*

Three quantities travel up the tree and are combined at every aggregation node. Each is an *accumulator*: a commutative monoid of values, a contribution of every leaf, and checks that the aggregation nodes and the Final Verifier apply to it (Table 6). The value of an aggregation node is the sum of the values of its non-null children, so by associativity and commutativity the value of every node is the sum of the contributions of the leaves below it, whatever the shape of the subtree. This independence of the shape is essential: the key is derived before the tree is built, so the quantities it is checked against must not depend on how the leaves are grouped. A hash-tree accumulation $\boldsymbol{c} = H(\boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3)$, for instance, would force the prover to fix the tree before deriving χ.

Accumulator	**Monoid**	**Leaf contribution**	**Child check**	**Root check**
leaf counter κ	$(\mathbb{F}, +)$	1	$\kappa \in [1, 2^B)$	$\kappa \in [1, 2^B)$
challenge accumulator $\boldsymbol{c}$	$(\mathbb{Z}_p^n, +)$	$h(\tau, \mathrm{com})$	—	$\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$
bus partial sum s	$(\mathbb{K}, +)$	$\mathrm{BusCont}_\chi(\mathcal{I}_\tau, w)$	—	$s + s_{\mathrm{glob}}(\boldsymbol{p}, \chi) = 0$

TABLE 6. The three accumulators of the recursion. $s_{\mathrm{glob}}(\boldsymbol{p}, \chi)$ is the LogUp sum of the global messages $\mathcal{G}(\boldsymbol{p})$ under the key χ (Section 3.3.1).

The values of the three accumulators at a node, together with the node's *type* τ, form its *recursive statement*

$$\sigma = (\tau, \kappa, \boldsymbol{c}, s),$$

where $\tau \in [M]$ is the AIR type for a leaf verifier, $\tau = M + 1$ marks an aggregation verifier, and $\tau = 0$ marks a null child. The counter is the only accumulator checked at every node: it bounds the number of leaves below any node by $2^B - 1$, which is the cardinality restriction required by the multiset hash (Section 6.2). The challenge accumulator and the bus partial sum are checked only at the root.

6.4.2. *Nodes of the Tree.*

AIR leaves. A leaf corresponds to a segment instance of one of the chips of ZisK, of *type* $\tau \in [M]$. Its proof establishes that its witness is *locally satisfying* —it satisfies the AIR, its fixed columns are those that the AIR prescribes, the AIR values that the AIR binds to the public statement equal the designated coordinates of $\boldsymbol{p}$, and its cached columns open the cached-column commitment carried in $\boldsymbol{p}$, which for the Rom AIR is root_{ROM}— and exposes the statement $(\boldsymbol{p}, \chi, s, \text{com})$, with s its bus contribution under χ and com its trace commitment. Exposing the commitment lets the leaf verifier above hash it into the challenge accumulator. The type is not part of the statement: it is determined by which AIR the leaf instantiates, and the leaf verifier above lifts it into its own statement.

Compressors. The STARK proofs of some AIRs are large (Section 7.3), and verifying them directly would make their leaf verifiers large as well. For these types, a *compressor* sits between the AIR leaf and its leaf verifier: a recursive verifier that verifies the AIR proof with the AIR's key and re-proves the same statement $(\boldsymbol{p}, \chi, s, \text{com})$ at a lower rate (Section 7). The compressor changes neither the statement nor its meaning; it only replaces the AIR proof by a smaller one.

Leaf verifiers. A leaf verifier sits above an AIR leaf, or above its compressor. There is one leaf verifier per AIR type: the leaf verifier of type τ verifies the AIR proof, or the compressor's proof, with a hard-coded key, and re-exposes its statement in the common format $(\boldsymbol{p}, \chi, \sigma, \mathsf{vk})$ of the tree, with recursive statement $\sigma = (\tau, 1, h(\tau, \text{com}), s)$: its type is the constant τ —fixed by which AIR the leaf verifier is for, not chosen by the prover—, its counter is 1, and its challenge contribution is the hash of its typed commitment, which it computes itself. The key vk it carries is not used by the leaf verifier: an aggregation verifier may need to verify a proof of *itself* as one of its children, and an AIR cannot hard-code its own verification key without a circular dependency at setup time. ZisK breaks this loop by treating that key as a public that is *propagated* through every level of the tree, with the Final Verifier closing the loop by fixing it to a hard-coded constant.

Aggregation verifiers. An aggregation verifier has three child slots. A slot holds either the proof of a child —a leaf verifier or another aggregation verifier— or a *null child*, a placeholder of type 0 that is not verified and contributes nothing; at least two slots hold proofs. The aggregation verifier reads each non-null child's type, checks that it is a leaf-verifier type with counter 1 or the aggregation type $M + 1$ with a counter in $[2, 2^B)$, and verifies the child accordingly: with the hard-coded leaf-verifier key of type τ for $\tau \in [M]$, or with the propagated key vk for $\tau = M + 1$. It verifies every child on a statement that carries its own $(\boldsymbol{p}, \chi, \mathsf{vk})$, and declares the type $M + 1$ and the sums of its children's accumulators. The aggregated challenge contribution is exactly the multiset-homomorphism step of $\mathcal{H}$: it is $\mathcal{H}$ of the union of the subtrees' multisets of typed commitments. Since every child is verified on its parent's $(\boldsymbol{p}, \chi, \mathsf{vk})$, every node of a valid tree carries the same public statement, key and verification key; and since the counters lie in $[1, 2^B)$, there are at least two non-null children and $3 \cdot 2^B < p$, every child's counter is strictly smaller than its parent's.

Final Verifier. The Final Verifier takes the topmost proof of the tree as its single child, which may not be a null child, and verifies it with the same dispatch as an aggregation verifier. It closes the recursion loop on vk: it hard-codes the actual verification key vk_{AGG} of the aggregation verifier and verifies its child on a statement that carries this constant, which binds the child's declared vk to it. It then performs the two global checks. The first, $s + s_{\text{glob}}(\boldsymbol{p}, \chi) = 0$, where s_{glob} is the bus contribution of the global messages $\mathcal{G}(\boldsymbol{p})$ (Section 6.3), which the Final Verifier computes itself, is the global bus-balancing condition under the key χ. The second, $\chi = \mathsf{DeriveKey}(\boldsymbol{p}, \boldsymbol{c})$, is *key correctness*: by the multiset homomorphism of $\mathcal{H}$, the value of $\boldsymbol{c}$ at the top of the tree is $\mathcal{H}$ of the multiset of typed commitments of the leaves, so this check confirms that χ was derived from the statement and the full set of commitments. After this point the recursive statement, the key and

the proof values are witnesses of the Final Verifier, and only the external statement $\boldsymbol{p}_{\text{ext}}$ remains visible to an external verifier.

Every execution has at least one Main segment and one Mem segment (Section 4.1.1), so the tree has at least two leaves and the root of the aggregation tree is an aggregation verifier; the Final Verifier nevertheless dispatches on the type, so it also accepts a single-leaf tree. Both global checks reduce to simple arithmetic on values propagated up the tree, with no need to inspect any individual AIR trace. This layered structure achieves a clean separation of responsibilities: AIR leaves prove only local constraint satisfaction; leaf verifiers convert proofs to a recursion-friendly format; aggregation verifiers accumulate the three running totals; and the Final Verifier enforces global bus balancing and key correctness. Because every node can proceed as soon as its children complete, the entire proving pipeline is fully parallelizable —no single AIR needs to handle the entire computation, eliminating the bottleneck of a monolithic AIR.

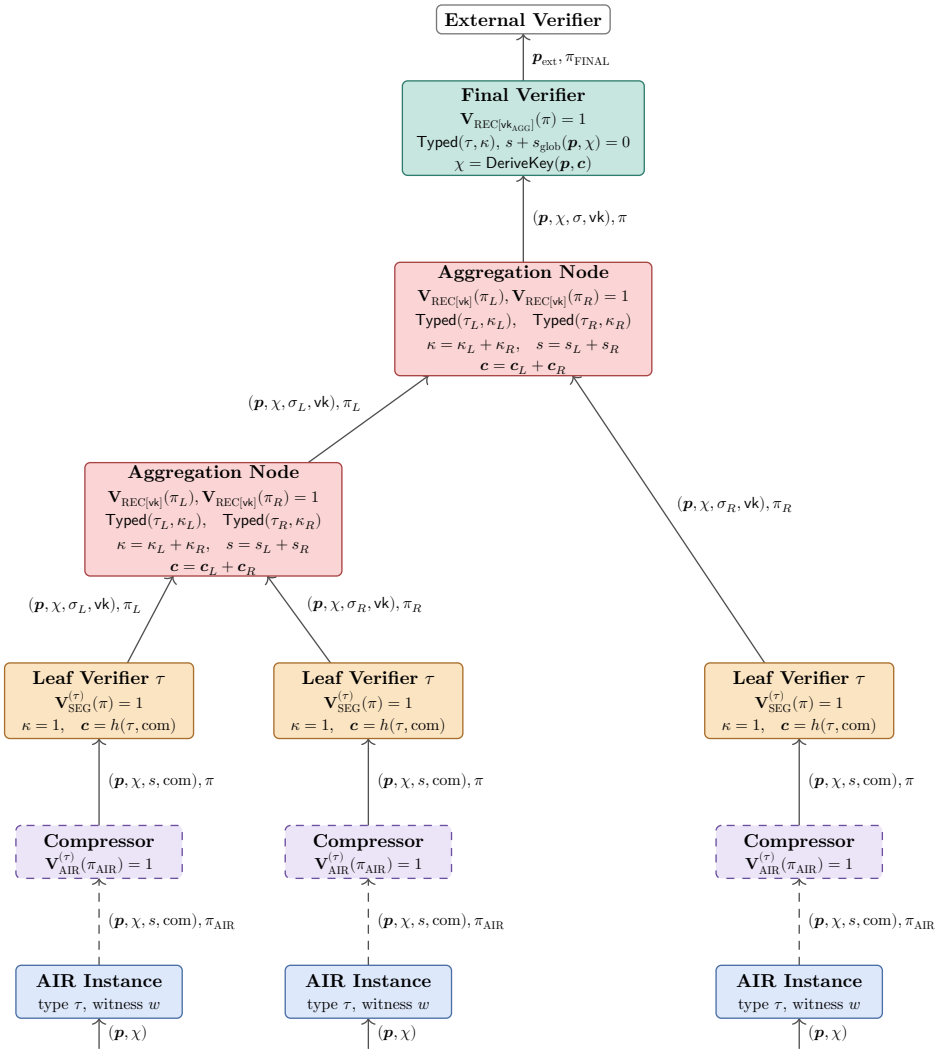


FIGURE 5. An aggregation tree with three AIR instances. Each arrow carries the statement and the proof that a node produces, which the node above takes as witness. The AIR proofs of some types are compressed: an optional compressor (dashed, purple) verifies the AIR proof and re-exposes the same statement, and the leaf verifier then verifies the compressor's proof instead; $\mathbf{V}^{(\tau)}_{\text{SEG}}$ denotes the verifier of the AIR proof or of its compressor, $\mathbf{V}_{\text{REC}}[\mathsf{vk}]$ the verification of a child according to its type, and Typed the check of the type and counter of a child. Aggregation verifiers are drawn with two children, whereas they have three slots, the unused ones holding null children.

6.5. **Security.**

The companion security analysis [Mas26] states the relation that every node of the tree attests to and proves what they establish together: the external verifier of the final proof is knowledge-sound for the execution relation $\mathcal{R}_{\text{exec}}$ (Definition 2.2), with an explicit error and under explicit assumptions on the primitives. It extracts, from any accepting final proof, a bounded collection of locally satisfying AIR witnesses that, together with the global messages, balance every bus, and decodes that collection into an execution of the committed program, following the bus contracts of Section 4.4.5 and the access log of Definition 4.2.

7. Proving Backend

The arithmetization framework of Section 3 is backend-independent by design, and the preceding sections have used the backend only through the abstract interface $\mathbf{V}(\mathsf{vk}, x, \pi)$. This section fixes the backend that ZisK actually uses —the commitment scheme, the low-degree test, the transcript and all their parameters— and maps every AIR of the recursion tree to the name under which it is registered in the Ethereum Foundation's `soundcalc` tool [Eth26], so that the soundness of each proof can be evaluated independently. The parameters below are those of the official ZisK setup.

7.1. The STARK Backend.

Every AIR of ZisK —the chips of Table 4 as well as the recursive verifiers of Section 6 — is proved with the same proving system: a univariate STARK [Ben+18b] following the DEEP-ALI paradigm, with FRI [Ben+18a] as low-degree test and Merkle trees as vector commitments, made non-interactive with the BCS transform [BCS16]. All arithmetic is over the Goldilocks field $\mathbb{F}$, and every verifier challenge and out-of-domain evaluation lies in the cubic extension $\mathbb{K}$ of Section 1. The exact protocol, as an interactive oracle proof and in its compiled form, and the analysis of its soundness are those of the security analysis [Mas26]; here we summarise it. For an AIR of trace height N, a proof proceeds in five stages.

1. **Trace commitment.** The witness columns are interpolated over a multiplicative subgroup of $\mathbb{F}^\times$ of size N, evaluated over a coset of a larger subgroup of size N/ρ, where ρ is the *rate* (the inverse of the *blowup factor*), and committed with a Merkle tree whose leaves are the rows of the evaluation matrix. The root of this tree is the *stage-1 commitment* of the proof, which enters the challenge derivation of Section 6.2. Fixed columns are committed once at setup, and the root of their tree is part of the AIR's verification key; the cached columns of the Rom AIR are committed once per program, and the root of their tree is the program commitment $\mathrm{root}_{\mathrm{ROM}}$ (Section 4.4.2).
2. **Bus commitment.** With the key $\chi = (\alpha, \gamma) \in \mathbb{K}^2$ of the bus argument, the prover commits, in a second Merkle tree, to the LogUp columns of Section 3.3.1: one fraction column for each group of interactions, the groups being chosen at setup to keep the degree of the constraints bounded, and the running-sum column. In ZisK this key is not sampled from the proof's own transcript but derived from the global challenge of Section 6.2, which every segment proof receives as a public input; the transcript of each AIR proof is seeded with it.
3. **Quotient.** The verifier samples a challenge, and the prover commits to the quotient of the random linear combination of all constraint polynomials (by powers of the challenge) by the vanishing polynomial of the trace domain. Intermediate columns keep the degree of each constraint below the bound allowed by the rate.
4. **DEEP evaluations.** The verifier samples an out-of-domain point $\xi \in \mathbb{K}$, and the prover sends the evaluations of all committed polynomials at ξ and at the shifted points required by the constraints.
5. **FRI.** The verifier samples two challenges with which the prover combines all committed polynomials into a single DEEP quotient, whose proximity to a low-degree polynomial is proved with FRI. FRI folds the evaluation domain by a factor of 8 per round (a final round may fold by 4), committing each folded layer with a Merkle tree, until the domain has 2^5 or 2^6 points, at which stage the remaining polynomial is sent in the clear. Before the query phase the prover must find a *grinding* nonce: a value whose Poseidon2 hash together with the last challenge has g leading zero bits, for a grinding parameter g. Finally,

q query positions are derived from the transcript, and the prover opens every committed tree at them.

Merkle trees and transcript. All Merkle trees and the Fiat–Shamir transcript use Poseidon2 [GKS23] over Goldilocks. Merkle trees have arity 4 (Poseidon2 of width 16), each row of a committed matrix is hashed into a leaf, and digests are 4 field elements, i.e. about 256 bits. To shorten proofs, Merkle authentication paths stop at a level close to the root —one with 4 or 16 nodes for most AIRs— and the proof carries that whole level. The transcript is a Poseidon2 sponge of width 16. For the AIR proofs it is seeded with the global challenge, which already depends on the public statement and on every stage-1 commitment (Section 6.2); for the recursive verifiers it is seeded with the verification key, the public inputs and the stage-1 commitment. It then absorbs every later commitment, the second-stage AIR values, the airgroup values and the evaluations before the corresponding challenges are squeezed.

7.2. Parameters and Security Level.

Rates, query counts and grinding are chosen per AIR so that every proof reaches 128 bits of security in the *Johnson-bound regime* (JBR) for Reed–Solomon proximity, i.e. assuming FRI soundness up to the Johnson radius, which is a proven regime and does not rely on proximity-gap conjectures. Given the rate ρ and the grinding g, the number of queries is the smallest q for which the query phase alone reaches 128 bits. Table 7 lists the resulting parameters by family of AIRs.

AIR	$\log_2 N$	ρ	q	g	FRI folding	d
ZisK AIRs (except below)	18–22	1/2	229–232	16	8 (4 last)	3
Rom	22	1/2	221	20	8	2
Keccakf	17	1/2	217	23	8	3
Poseidon2	17	1/4	114	16	8 (4 last)	4
Compressors	18–20	1/4	110	20	8 (4 last)	5
Leaf and aggregation verifiers	17	1/8	73	20	8	8
Final Verifier	16	1/32	43	22	16	8

TABLE 7. Backend parameters of ZisK: trace height N (per AIR in Table 4), rate ρ, number of FRI queries q, grinding bits g, FRI folding factor, and maximum constraint degree d. All AIRs use Goldilocks with its cubic extension, Poseidon2 Merkle trees of arity 4 with 256-bit digests, and LogUp for every bus.

With these parameters, `soundcalc` [Eth26] reports 128 bits of security in the Johnson-bound regime for every AIR of ZisK. Under the more conservative *unique-decoding regime* (UDR), which only assumes FRI soundness up to half the minimum distance, it reports between 111 and 113 bits for the AIRs at rate 1/2, 93 bits for Poseidon2, 94 bits for the compressors, 80 bits for the leaf and aggregation verifiers and 63 bits for the Final Verifier. ZisK therefore claims 128-bit security in the Johnson-bound regime.

Remark (Proof Size and Compressors). *The high rate and large query count of the AIR proofs make them fast to produce but large, especially for the AIRs with the widest traces. The compressor nodes of Section 6 verify such a proof and re-prove its statement at rate* 1/4 *with* 110 *queries, which is what keeps the leaf verifier within* 2^{17} *rows.*

7.3. Topology of All Proof-System Calls.

Table 8 lists every AIR for which a STARK proof is produced in a ZisK proof, its role in the aggregation tree of Section 6, the relation it proves in the security analysis [Mas26], and the number of instances an execution produces. An execution with L leaves has between $\lceil (L-1)/2 \rceil$ and $L-1$ aggregation verifiers, since each has two or three non-null children; with a single leaf it has none, and the Final Verifier verifies the leaf verifier directly. All auxiliary arguments —memory, lookups, permutations, range checks and continuations— are realised as bus interactions inside the AIR leaves (Section 4.4), not as separate proof-system invocations.

Node	**AIR**	**Relation**	**Instances**
AIR leaf	each of the 35 AIRs of Table 4	$\mathcal{R}_{\text{AIR}}^{(\tau)}$	one per segment of the AIR (Definition 5.1)
compressor	Keccakf, Sha256f, Blake2br, ArithEq, ArithEq384, DmaPrePost	compressor relation of $\Pi_{\text{AIR}}^{(\tau)}$	one per leaf of these AIRs
leaf verifier	one per AIR	$\mathcal{R}_{\text{LEAF}}^{(\tau)}$	one per leaf
aggregation verifier	one	$\mathcal{R}_{\text{AGG}}$	between $\lceil (L-1)/2 \rceil$ and $L-1$
Final Verifier	one	$\mathcal{R}_{\text{FINAL}}$	1

TABLE 8. Topology of all proof-system calls in ZisK, for an execution with L leaves. The relations are those of the security analysis [Mas26].

Every AIR of Table 8 is registered in `soundcalc` [Eth26]: each AIR under its own name (e.g. `Main`, `Mem`, `Keccakf`), each compressor as `<AIR>-compressor` (e.g. `Keccakf-compressor`), the aggregation verifier as `Recursive2`, and the Final Verifier as `Final`.

The leaf verifiers of all AIRs share the STARK parameters and constraint system of the aggregation verifier and differ from it only in their fixed columns, which hard-code the key of the AIR they verify; the `soundcalc` entry `Recursive2` therefore covers both kinds of node. Finally, the Final Verifier's proof can be wrapped into a pairing-based SNARK over BN254 for cheap on-chain verification; this wrapping step is outside the scope of this document and of `soundcalc`.

Acknowledgments

This paper draws on the designs presented in the OpenVM [Ope25] and Nexus [Nex24] whitepapers, which served as important sources of inspiration. We also acknowledge RISC Zero [RIS23] as a pioneering zkVM and SP1 [Suc24] for advancing the practical deployment of RISC-V-based proving systems.

We extend our gratitude to the Polygon zkEVM and Plonky3 [Pol24] teams for their foundational work in zero-knowledge proving systems, as well as to the RISC-V community [RIS19] for providing a robust architecture that enables the zkVM model.

References

[Ajt96] Miklós Ajtai. Generating Hard Instances of Lattice Problems (Extended Abstract). In *28th Annual ACM Symposium on Theory of Computing*, pages 99–108. ACM Press, May 1996. doi:10.1145/237814.237838.

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive Oracle Proofs. In *TCC 2016-B: 14th Theory of Cryptography Conference, Part II*, volume 9986 *of Lecture Notes in Computer Science*, pages 31–60. Springer, Berlin, Heidelberg, 2016. doi:10.1007/978-3-662-53644-5_2.

[Bel+23] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina. Circom: A Circuit Description Language for Building Zero-Knowledge Applications. *IEEE Transactions on Dependable and Secure Computing*, 20(6):4733–4751, 2023. doi:10.1109/TDSC.2022.3232813.

[Ben+18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018: 45th International Colloquium on Automata, Languages and Programming*, volume 107 *of Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17. Schloss Dagstuhl, July 2018. doi:10.4230/LIPIcs.ICALP.2018.14.

[Ben+18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018. URL: https://eprint.iacr.org/2018/046.

[Cla+03] Dwaine E. Clarke, Srinivas Devadas, Marten van Dijk, Blaise Gassend, and G. Edward Suh. Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. In Chi-Sung Laih, editor, *Advances in Cryptology – ASIACRYPT 2003*, volume 2894 *of Lecture Notes in Computer Science*, pages 188–207. Springer, Berlin, Heidelberg, November 2003. doi:10.1007/978-3-540-40061-5_12.

[DP23] Benjamin E. Diamond and Jim Posen. Succinct Arguments over Towers of Binary Fields. Cryptology ePrint Archive, Report 2023/1784, 2023. URL: https://eprint.iacr.org/2023/1784.

[Ele22] Electric Coin Company. Halo2. 2022. URL: https://github.com/zcash/halo2.

[Eth26] Ethereum Foundation. soundcalc. GitHub repository, 2026. URL: https://github.com/ethereum/soundcalc.

[FS87] Amos Fiat and Adi Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology*

– *CRYPTO'86*, volume 263 *of Lecture Notes in Computer Science*, pages 186–194. Springer, Berlin, Heidelberg, August 1987. doi:10.1007/3-540-47721-7_12.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A Faster Version of the Poseidon Hash Function. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23: 14th International Conference on Cryptology in Africa*, volume 14064 *of Lecture Notes in Computer Science*, pages 177–203. Springer, Cham, July 2023. doi:10.1007/978-3-031-37679-5_8.

[GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a Turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Report 2021/1063, 2021. URL: https://eprint.iacr.org/2021/1063.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. URL: https://eprint.iacr.org/2019/953.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022. URL: https://eprint.iacr.org/2022/1530.

[Hem+25] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh, and Ron D. Rothblum. Jagged Polynomial Commitments (or: How to Stack Multilinears). Cryptology ePrint Archive, Report 2025/917, 2025. URL: https://eprint.iacr.org/2025/917.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Report 2024/278, 2024. URL: https://eprint.iacr.org/2024/278.

[Lit24] Lita Foundation. Valida: a STARK-based virtual machine focusing on code reuse. 2024. URL: https://github.com/valida-xyz/valida.

[Mas26] Héctor Masip-Ardevol. Security of ZisK. ZisK Core Team technical report, 2026. URL: https://github.com/0xPolygonHermez/zisk-whitepaper.

[Nex24] Nexus. The Nexus zkVM. 2024. URL: https://nexus.xyz/.

[Ope25] OpenVM Contributors. OpenVM Whitepaper. 2025. URL: https://openvm.dev/whitepaper.pdf.

[Pol24] Polygon. Plonky3. 2024. URL: https://github.com/Plonky3/Plonky3.

[RIS19] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. Technical report Version 20191213, 2019. URL: https://riscv.org/wp-content/uploads/2019/12/riscv-spec-20191213.pdf.

[RIS23] RISC Zero. RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity. 2023. URL: https://www.risczero.com/.

[RIS24] RISC-V International. RISC-V ELF psABI Specification. Versioned living document, 2024. URL: https://github.com/riscv-non-isa/riscv-elf-psabi-doc.

[Suc24] Succinct Labs. SP1: A performant, open-source zkVM. 2024. URL: https://github.com/succinctlabs/sp1.

[Zis25] ZisK. Secure Challenge Derivation in ZisK. HackMD technical note, 2025. URL: https://hackmd.io/SXKiWASTRu2QYgy7B-NGmg.

Appendix

A. Arguments

This appendix gives the formal statements of the argument types used in ZisK's arithmetization framework. All are realised through the same bus-balancing primitive of Section 3.4 —one or more sender interactions with positive multiplicity, one or more receiver interactions with negative multiplicity, on a shared bus— but they differ in the semantic relationship they enforce on the messages.

Throughout this appendix we fix the chips $(\mathcal{C}_i, \mathcal{I}_i)$, $i \in [M]$, of ZisK, with witnesses $\{(\boldsymbol{T}_i, \boldsymbol{a}_i)\}_{i=1}^{M}$, and we write $m(r), \boldsymbol{e}(r) \in \mathbb{F}^{\ell}$ for the row-r evaluation of an interaction $(b, m, \boldsymbol{e})$ under its witness.

Permutation Argument.

A *permutation argument* enforces that two AIRs observe the same multiset of messages on a shared bus. It is the natural primitive whenever data is reordered —e.g., between unsorted and sorted views of the same memory accesses— but not modified.

Definition (Permutation Argument). Let $\iota_1 = (b, m_1, \boldsymbol{e}_1) \in \mathcal{I}_{i_1}$ and $\iota_2 = (b, -m_2, \boldsymbol{e}_2) \in \mathcal{I}_{i_2}$ be a sender/receiver pair of interactions of common length ℓ on a shared bus b, with boolean multiplicities $m_1, m_2 \in \{0, 1\}$. The pair (ι_1, ι_2) realises a *permutation argument* in the sense that the joint contribution of ι_1 and ι_2 to bus b vanishes if and only if

$$\left\{\boldsymbol{e}_1(r) : r \in \left[N_{i_1}\right], m_1(r) = 1\right\} = \left\{\boldsymbol{e}_2(r) : r \in \left[N_{i_2}\right], m_2(r) = 1\right\}$$

as multisets.

Lookup Argument.

A *lookup argument* enforces that every message sent on the bus by a "query" AIR also appears as a row of a "table" AIR, with the table free to receive each query any number of times.

Definition (Lookup Argument). Let $\iota_q = \left(b, m_q, \boldsymbol{e}_q\right) \in \mathcal{I}_{i_q}$ be a *query* interaction with boolean multiplicity $m_q \in \{0, 1\}$, and $\iota_t = (b, -m_t, \boldsymbol{e}_t) \in \mathcal{I}_{i_t}$ a *table* interaction with $m_t(r) \geq 0$ counting how many queries land on row r of the table. Both interactions have common length ℓ. The pair $\left(\iota_q, \iota_t\right)$ realises a *lookup argument* in the sense that their joint contribution to bus b vanishes if and only if

$$\forall r \in \left[N_{i_q}\right] \text{ with } m_q(r) = 1 : \exists r' \in \left[N_{i_t}\right] \text{ s.t. } \boldsymbol{e}_q(r) = \boldsymbol{e}_t(r').$$

Lookup arguments underpin most of ZisK's table-driven constraints: instruction decoding, byte and limb decompositions, and any check whose efficient form is "the output of a sub-chip must belong to a precomputed table".

Range Check Argument.

A *range check argument* enforces that an expression takes values in a bounded interval. It is a specialisation of the lookup argument against a fixed interval table, but it is exposed as a first-class primitive because its receiving side is a shared system table rather than an AIR-specific one.

Definition (Range Check Argument). Fix integers $x \leq y$ and let b be the bus designated for the range check on $[x, y]$. Let $\iota = (b, m, e) \in \mathcal{I}_i$ be an interaction of length $\ell = 1$ with boolean multiplicity $m \in \{0, 1\}$, paired with a receiver interaction $\iota_t = (b, -m_t, e_t)$ on a dedicated table AIR whose trace materialises the set $\{x, x + 1, ..., y\}$ and whose multiplicity $m_t(r) \geq 0$ counts how many incoming queries land on each value. The pair (ι, ι_t) realises a *range check argument* on $[x, y]$ in the sense that their joint contribution to bus b vanishes if and only if

$$\forall r \in [N_i] \;\; \text{with} \;\; m(r) = 1 : e(r) \in [x, y].$$

In practice ZisK shares each range-check table across all the chips: a single $[x, y]$ table absorbs queries from every AIR that needs that range, so the marginal cost of a range check is one row of querying plus one row of multiplicity bookkeeping in the shared table.

Copy Argument.

A *copy argument* asserts that a designated set of cells in a single AIR's trace hold equal values. It is the wiring primitive used inside an AIR to relate values that must coincide but live in different rows or columns —for instance, feeding the output of one row back as the input of a later row, or forcing two columns to agree on a shared intermediate value.

Definition (Copy Argument). Let $\mathcal{C}$ be an AIR with witness $(\boldsymbol{T}, \boldsymbol{a})$, and let $C = \{(r_1, j_1), ..., (r_q, j_q)\}$ be a finite set of *cell coordinates* in $\boldsymbol{T}$, each (r, j) identifying the cell at row r of column j. Let $\{C_1, ..., C_t\}$ be a partition of C into *equivalence classes*. A *copy argument* for $(C, \{C_\ell\})$ enforces:

$$\forall \ell \in [t], \forall (r, j), (r', j') \in C_\ell : T_{j,r} = T_{j',r'}.$$

Operationally, a copy argument is realised as a permutation argument applied class-by-class: for each C_ℓ, the values at all cells in C_ℓ are sent on a dedicated bus and required to coincide as multisets with the same values shifted by an arbitrary cycle on C_ℓ —this forces every pair of cells in the class to agree.

B. Instruction Set

This appendix lists every opcode of the ZisK ISA, grouped by the chip that proves it (Table 4). It transcribes the reference implementation `zisk_ops.rs` and is the normative definition of the operation $(c, \text{flag}) = \text{op}(a, b)$ used by the step semantics of Section 2.4.

We write $a_{32} = a \bmod 2^{32}$ for the low word of a; $\text{sext}_k(x)$ for the sign extension of a k-bit value x to 64 bits; $<_u, \leq_u$ and $<_s, \leq_s$ for unsigned and signed (two's complement) comparison; and $[\varphi] \in \{0, 1\}$ for the indicator of φ. Arithmetic on 64-bit words is modulo 2^{64} and on 32-bit words modulo 2^{32}, unless stated otherwise. Each entry reads "mnemonic (code): c; flag", with flag $= 0$ when omitted. The 32-bit (`_w`) opcodes depend only on the low 32 bits of their operands. Opcodes marked † are part of the ISA but are not emitted by the transpiler of Section 2.1.

Internal Opcodes (Main).

`flag (0x00)` $c = 0$; flag $= 1$. Used for unconditional jumps.

`copyb (0x01)` $c = b$.

`halt (0xff)` the invalid-instruction sentinel (Section 2.1); it cannot appear in a valid execution.

Binary Opcodes (Binary, BinaryAdd).

64-bit:

`minu (0x02)`, `min (0x03)` $c = a$ if $a <_u b$ (resp. $a <_s b$), and $c = b$ otherwise.

`maxu (0x04)`, `max (0x05)` $c = a$ if $b <_u a$ (resp. $b <_s a$), and $c = b$ otherwise.

`ltu (0x06)`, `lt (0x07)` $c = [a <_u b]$, resp. $[a <_s b]$; flag $= c$.

`eq (0x09)` $c = [a = b]$; flag $= c$.

`add (0x0a)` $c = a + b$. Proved by Binary or by the dedicated BinaryAdd chip.

`sub (0x0b)` $c = a - b$.

`leu† (0x0c)`, `le† (0x0d)` $c = [a \leq_u b]$, resp. $[a \leq_s b]$; flag $= c$.

`and (0x0e)`, `or (0x0f)`, `xor (0x10)` bitwise conjunction, disjunction and exclusive disjunction of a and b.

32-bit:

`minu_w (0x12)`, `min_w (0x13)` $c = \text{sext}_{32}(a_{32})$ if $a_{32} <_u b_{32}$ (resp. $<_s$), and $c = \text{sext}_{32}(b_{32})$ otherwise.

`maxu_w (0x14)`, `max_w (0x15)` $c = \text{sext}_{32}(a_{32})$ if $b_{32} <_u a_{32}$ (resp. $<_s$), and $c = \text{sext}_{32}(b_{32})$ otherwise.

`ltu_w† (0x16)`, `lt_w† (0x17)` $c = [a_{32} <_u b_{32}]$, resp. $[a_{32} <_s b_{32}]$; flag $= c$.

`eq_w† (0x19)` $c = [a_{32} = b_{32}]$; flag $= c$.

`add_w (0x1a)`, `sub_w (0x1b)` $c = \text{sext}_{32}((a_{32} + b_{32}) \bmod 2^{32})$, resp. $\text{sext}_{32}((a_{32} - b_{32}) \bmod 2^{32})$.

`leu_w† (0x1c)`, `le_w† (0x1d)` $c = [a_{32} \leq_u b_{32}]$, resp. $[a_{32} \leq_s b_{32}]$; flag $= c$.

Binary Extension Opcodes (BinaryExtension).

`sll (0x21)`, `srl (0x22)`, `sra (0x23)` a shifted by $b \bmod 64$ bits: left, logical right, and arithmetic right, respectively.

`sll_w (0x24)`, `srl_w (0x25)`, `sra_w (0x26)` $c = \text{sext}_{32}$ of a_{32} shifted by $b \bmod 32$ bits: left, logical right, and arithmetic right, respectively.

`signextend_b` (`0x27`), `signextend_h` (`0x28`), `signextend_w` (`0x29`) $c = \mathrm{sext}_8(b \bmod 2^8)$, $\mathrm{sext}_{16}(b \bmod 2^{16})$, resp. $\mathrm{sext}_{32}(b_{32})$; a is ignored.

Arithmetic Opcodes (Arith).

`mul` (`0xb4`), `mulu†` (`0xb0`) $c = a \cdot b \bmod 2^{64}$ (the result is the same for the signed and unsigned readings).

`muluh` (`0xb1`), `mulsuh` (`0xb3`), `mulh` (`0xb5`) $c =$ the high 64 bits of the 128-bit product $a \cdot b$, reading a and b as unsigned and unsigned, signed and unsigned, and signed and signed, respectively.

`mul_w` (`0xb6`) $c = \mathrm{sext}_{32}(a_{32} \cdot b_{32} \bmod 2^{32})$.

`divu` (`0xb8`), `remu` (`0xb9`) $c = \lfloor a/b \rfloor$, resp. $a \bmod b$, unsigned. If $b = 0$: $c = 2^{64} - 1$, resp. $c = a$, and flag $= 1$.

`div` (`0xba`), `rem` (`0xbb`) signed quotient rounded toward zero, resp. remainder with the sign of a. If $b = 0$: $c = 2^{64} - 1$, resp. $c = a$, and flag $= 1$. The overflow case $-2^{63}/-1$ gives $c = -2^{63}$, resp. $c = 0$.

`divu_w` (`0xbc`), `remu_w` (`0xbd`) $c = \mathrm{sext}_{32}$ of the unsigned 32-bit quotient, resp. remainder, of a_{32} and b_{32}. If $b_{32} = 0$: $c = 2^{64} - 1$, resp. $c = \mathrm{sext}_{32}(a_{32})$, and flag $= 1$.

`div_w` (`0xbe`), `rem_w` (`0xbf`) $c = \mathrm{sext}_{32}$ of the signed 32-bit quotient rounded toward zero, resp. remainder with the sign of a_{32}, of a_{32} and b_{32}. If $b_{32} = 0$: $c = 2^{64} - 1$, resp. $c = \mathrm{sext}_{32}(a_{32})$, and flag $= 1$.

Public Output and Hint Opcodes (Main).

`pubout` (`0x30`) $c = b$. Declares the public output with index $a \in [0, 32)$ to be b (Section 2.3.8). Dispatched by Main on the Operation Bus, where it is matched against the public statement (Section 6.3).

`fcall_param` (`0xf6`) $c = b$. Appends to the parameters of the next host call either the value b (if $a = 1$) or the a words stored at address b.

`fcall` (`0xf7`) $c = b$. Requests the host computation identified by a on the parameters passed so far; its result becomes the hint stream.

`fcall_get` (`0xf8`) $c = b$, where b is the value read from the free-input port, i.e. the next word of the hint stream.

The three hint opcodes only affect the host state, and Main proves them as `copyb` (Section 2).

Precompile Opcodes.

For every precompile, b is the address P of an 8-byte-aligned parameter block and $a = 0$; $c = 0$ and flag $= 0$ unless stated otherwise. All multi-word values are sequences of 64-bit words in little-endian order (least significant word first) and every pointer in a parameter block is 8-byte aligned. The memory effect of each opcode is described below; $\mathrm{mem}[x, x + n)$ denotes the n bytes at address x, and $P_i \coloneqq \mathrm{mem}[P + 8i]_8$ the i-th word of the parameter block.

`keccak` (`0xf1`, Keccakf) $\mathrm{mem}[P, P + 200)$ is replaced by its image under the Keccak-f[1600] permutation.

`sha256` (`0xf9`, Sha256f) the 256-bit state at P_0 is replaced by its SHA-256 compression with the 512-bit block at P_1.

`poseidon2` (`0xe1`, Poseidon2) the 16 words at P, read as Goldilocks elements, are replaced by their image under the Poseidon2 permutation of width 16.

`blake2` (`0xea`, Blake2br) one BLAKE2b round, with round index P_0, on the 16-word state at P_1 and the 16-word message block at P_2; the state is updated in place.

`add256` (`0xf0`, Add256) with operands at P_0, P_1, a carry in $P_2 \in \{0,1\}$ and result pointer P_3, writes $(x + y + P_2) \bmod 2^{256}$ at P_3 and returns the carry out in $c \in \{0,1\}$.

`arith256` (`0xf2`, ArithEq) with 256-bit operands x, y, z at P_0, P_1, P_2, writes the low and high 256-bit halves of $x \cdot y + z$ at P_3 and P_4.

`arith256_mod` (`0xf3`, ArithEq) with 256-bit operands x, y, z, m at $P_0, ..., P_3$, writes $(x \cdot y + z) \bmod m$ at P_4.

`arith384_mod` (`0xe2`, ArithEq384) as `arith256_mod`, with 384-bit operands.

`secp256k1_add` (`0xf4`), `secp256r1_add` (`0xe8`), `bn254_curve_add` (`0xfa`) (ArithEq) with affine points p_1, p_2 at P_0, P_1, replaces p_1 by $p_1 + p_2$ on the named curve, computed with the chord formula.

`secp256k1_dbl` (`0xf5`), `secp256r1_dbl` (`0xe9`), `bn254_curve_dbl` (`0xfb`) (ArithEq) the affine point p at P itself is replaced by $2p$, computed with the tangent formula.

`bn254_complex_add` (`0xfc`), `bn254_complex_sub` (`0xfd`), `bn254_complex_mul` (`0xfe`) (ArithEq) with elements f_1, f_2 of the quadratic extension of the BN254 base field at P_0, P_1, replaces f_1 by $f_1 + f_2$, $f_1 - f_2$, resp. $f_1 \cdot f_2$.

`bls12_381_curve_add` (`0xe3`), `bls12_381_curve_dbl` (`0xe4`) (ArithEq384) as the 256-bit curve operations, on BLS12-381 with 384-bit coordinates.

`bls12_381_complex_add` (`0xe5`), `bls12_381_complex_sub` (`0xe6`), `bls12_381_complex_mul` (`0xe7`) (ArithEq384) as the BN254 extension-field operations, over the BLS12-381 base field.

DMA Opcodes (DMA chips).

Unlike the other precompiles, the DMA opcodes take a and b as plain values. The byte count n is the word at the fixed system address `0xA000_0F00` for `dma_memcpy` and `dma_memcmp`, and the extended argument of the instruction (its `j1` field) for the `x` variants. flag $= 0$ always.

`dma_memcpy` (`0xd0`), `dma_xmemcpy` (`0xd6`) $\mathrm{mem}[a, a+n) \leftarrow \mathrm{mem}[b, b+n)$, with the semantics of a copy through a temporary buffer when the ranges overlap; $c = a$.

`dma_memcmp` (`0xd1`), `dma_xmemcmp` (`0xd7`) $c =$ the difference $\mathrm{mem}(a+k) - \mathrm{mem}(b+k)$, as a signed 64-bit value, at the first offset $k < n$ where the two ranges differ, and $c = 0$ if they are equal; memory is unchanged.

`dma_xmemset` (`0xd9`) $\mathrm{mem}[a, a+b)$ is filled with the byte given by the extended argument; $c = a$.

`dma_inputcpy` (`0xd2`) copies the next $b/8$ words of the hint stream to $\mathrm{mem}[a, a+b)$, where b is a multiple of 8; $c = a$.

The opcodes `0xda`, `0xdb`, `0xdc` and `0xdd` are internal to the DMA chips: they label the aligned, unaligned, head and tail phases into which a bulk operation is decomposed, and never appear in a program.